

嵌入式就业冲刺-重点复盘-总结-补充

《内部资料，拒绝分享，导致面试内卷》 - 上官可编程 - 2024-10-31

嵌入式就业冲刺-重点复盘-总结-补充

一. C语言阶段

1. 高频出现的关键字篇 - 面试常见

1.1 define关键字

1.1.1 定义简单的宏

1.1.2 定义带参数的宏

示例：定义求平方

示例：定义一年多少天

示例：求圆面积和周长

示例：返回小值

示例：宏与函数的比较

示例：**字符串化运算符：**

1.1.3 宏的取消定义（#undef）

1.1.4 修饰头文件

1.2 typedef关键字-别名作用

1.2.1 为基本数据类型定义别名

1.2.2 为复杂的数据类型定义别名

1.2.3 为数组类型定义别名

1.2.4 为指针类型定义别名

1.2.5 为函数指针类型定义别名

1.2.6 特点总结

1.3 extern关键字

1.3.1 全局变量的外部链接

1.3.2 函数的外部链接

1.3.3 头文件中的使用

1.3.4 声明和定义的区别

1.4. static关键字

1.4.1 修饰局部变量

1.4.2 修饰全局变量

1.4.3 static修饰函数

1.4.4 static修饰的变量存放在哪里

1.5 const关键字

1.5.1 定义常量

1.5.2 修饰函数参数

1.5.3 修饰函数返回值

1.5.4 修饰指针

1.5.5 修饰数组

1.6 sizeof关键字

1.6.1 基本用法

1.6.2 特点

1.6.3 示例代码

1.6.4 sizeof和strlen区别

1.7 register关键字

1.7.1 基本用法

1.7.2 示例代码

1.8 volatile关键字

1.8.1 基本用法

1.8.2 示例代码

- 1.9 enum关键字
 - 1.9.1 基本用法
 - 1.9.2 特点和注意事项
 - 1.9.3 示例代码
- 1.10 auto关键字
 - 1.10.1 基本用法 (历史用法)
 - 1.10.2 现代用法
 - 1.10.3 C++ 中的 `auto` 关键字
 - 1.10.4 C11 中的 `auto` 关键字
- 1.11 C语言预处理
 - 1.11.1 `#include`
 - 1.11.2 `#define`
 - 1.11.3 `#undef`
 - 1.11.4 `#ifdef`, `#ifndef`, `#if`, `#else`, `#elif`, `#endif`
 - 1.11.5 `#pragma` 这里列出
 - 1.11.6 `#line`
 - 1.11.7 `#error`
 - 1.11.8 `#warning`
- 2. 指针篇 - 更多是辅助梳理总结
 - 2.1 指针是什么有什么作用
 - 2.1.1 动态内存管理
 - 2.1.2 函数参数传递改值
 - 2.1.3 性能优化
 - 2.1.4 实现高级数据结构
 - 2.1.5 字符串操作
 - 2.1.6 函数返回多个值
 - 2.1.7 灵活性和控制
 - 2.1.8 与C语言库的兼容性
 - 2.1.9 优化数组操作
 - 2.1.10 接口硬件
 - 2.2 指针和变量
 - 2.2.1 指针和变量的关系**
 - 2.3 指针数组
 - 2.3.1 它是一个数组，数组的每一项都是一个指针。
 - 2.4 数组指针
 - 2.4.1 它是一个指针，指向一个固定大小的数组
 - 2.4.2 数组指针和二维数组
 - 2.5 指针函数-返回指针的函数
 - 2.5.1 返回局部变量的指针 (不推荐)
 - 2.5.2 返回动态分配的内存地址
 - 2.5.3 返回静态变量的指针
 - 2.6 函数指针-回调函数的应用
 - 2.6.1 使用函数指针
 - 2.6.2 函数指针作为参数-回调函数
 - 2.7 指针和二维数组
 - 2.7.1 指针与二维数组的关系
 - 2.7.2 使用数组名访问二维数组**
 - 2.7.3 使用指向数组的指针访问二维数组**
 - 2.7.4 动态分配二维数组**
 - 2.8 二级指针
 - 2.8.1 定义二级指针
 - 2.8.2 初始化二级指针
 - 2.8.3 使用二级指针
 - 2.8.4 动态内存分配
 - 2.9 指针和结构体

- 2.9.1 使用指针和结构体管理学生信息
- 2.10 野指针 - 面试常见
 - 2.10.1 野指针的成因
 - 2.10.2 野指针的危害
 - 2.10.3 避免野指针的措施
 - 2.10.4 演示野指针
 - 2.10.5 安全的代码示例
- 2.11 内存泄露 - 面试常见
 - 2.11.1 内存泄漏的成因
 - 2.11.2 内存泄漏的危害
 - 2.11.3 避免内存泄漏的措施
 - 2.11.4 演示内存泄漏
 - 2.11.5 安全的代码示例
- 3. 高频出现的编程题篇 - 其实可以直接背下记住
 - 3.1 冒泡排序
 - 3.1.1 算法步骤
 - 3.1.2 代码实现
 - 3.1.3 冒泡排序的特点
 - 3.2 选择排序
 - 3.2.1 算法步骤
 - 3.2.2 代码实现
 - 3.2.3 选择排序的特点
 - 3.3 字符串拷贝
 - 3.3.1 代码实现
 - 3.3.2 注意事项
 - 3.4 字符串分割
 - 3.4.1 代码实现
 - 3.4.2 代码解释
 - 3.5 链表逆序
 - 3.5.1 链表逆序逻辑
 - 3.5.2 逐行代码及注释
 - 3.7 把数字转成字符串
 - 3.8 编写一个函数实现字符串的翻转
 - 3.9 指针定义汇总-笔试会出现
- 4. 其他
 - 4.1 补码
 - 4.1.1 补码的定义
 - 4.1.2 补码的优点
 - 4.1.3 补码的计算
 - 4.1.4 溢出和范围
 - 4.1.5 示例
 - 4.2 反码-了解一下
 - 4.2.1 反码的定义
 - 4.2.2 反码的计算方法
 - 4.2.3 补码反码的关系
 - 4.3 结构体大小计算
 - 4.3.1 结构体大小的影响因素
 - 4.3.2 计算结构体大小
 - 4.3.3 强制结构体对齐
 - 4.4 结构体位域
 - 4.4.1 位域的定义和使用
 - 4.4.2 位域的特性
 - 4.4.3 注意事项
 - 4.4.4 位域的大小和对齐
 - 4.5 联合体大小计算
 - 4.5.1 联合体的定义

- 4.5.2 联合体大小的计算
- 4.5.3 联合体的特性
- 4.5.4 注意事项
- 4.6 大小端字节序
 - 4.6.1 大端字节序 (Big-endian)
 - 4.6.2 小端字节序 (Little-endian)
 - 4.6.3 大小端字节序的影响
 - 4.6.4 如何检测大小端
 - 4.6.6 字节序转换函数
- 4.7 C语言数据存放位置-重要-面试常问
 - 4.7.1 存储在栈上的数据
 - 4.7.2 存储在堆上的数据
 - 4.7.3 区别
- 4.8 引用和指针的区别

一. C语言阶段

1. 高频出现的关键字篇 - 面试常见

1.1 define关键字

重要提醒：笔试的时候，宏的拼写用大写字母！虽然语法不会报错，但会留下不专业的印象。

`#define` 是 C 语言中的一个预处理指令，而不是一个编译时的关键字。

它用于在预处理阶段向源代码中定义宏。

宏是源代码中的一个标识符，被替换为另一个字符串，这在编译之前发生。

下面是关于 `#define` 的一些基本用法和细节：

1.1.1 定义简单的宏

示例：定义一个常量

```
#include <stdio.h>

// 定义一个简单的宏
#define PI 3.14159

int main() {
    double radius = 10.0;
    double area = PI * radius * radius;
    printf("The area of the circle is: %f\n", area);
    return 0;
}
```

这个宏定义使得在代码中每次出现 `PI` 时，预处理器都会将其替换为 `3.14159`。

两个好处（面试过程中可能会问）：

- 批量修改，只要修改PI后面的3.14.159，那么后续代码中都会被修改，如果没有用宏定义，有多少个地方出现3.14159就要修改多少次

- 程序可阅读性更高，比如 `setArea(100,200)`; `setArea`函数假设是获取某个区域，100,200怪怪的，如果define `WIDTH 100`，`define HEIGHT 200`，那么在函数调用的时候，`setArea(WIDTH, HEIGHT)`;看着就非常直观。

1.1.2 定义带参数的宏

!!!!!! 示例代码高频出现笔试中!!!!!!

示例：定义求平方

```
#include <stdio.h>
// 定义一个带参数的宏
#define SQUARE(x) ((x) * (x))

int main() {
    int num = 5;
    int square = SQUARE(num);
    printf("The square of %d is %d\n", num, square);
    return 0;
}
```

这个宏定义使得 `SQUARE(a)` 会被替换为 `((a) * (a))`。

示例：定义一年多少天

```
#include <stdio.h>

// 定义一个带参数的宏，计算一年的天数
#define DAYS_IN_YEAR(isLeapYear) ((isLeapYear) ? 366 : 365)

int main() {
    int isLeapYear = 1; // 假设这是闰年
    int days = DAYS_IN_YEAR(isLeapYear);
    printf("A leap year has %d days.\n", days);

    isLeapYear = 0; // 假设这是平年
    days = DAYS_IN_YEAR(isLeapYear);
    printf("A common year has %d days.\n", days);

    return 0;
}
```

在这个代码中，`DAYS_IN_YEAR` 宏接受一个参数 `isLeapYear`，如果该参数为真（非零值），则返回366天，表示闰年；如果为假（零值），则返回365天，表示平年。

当你编译并运行这段代码时，它会输出：

```
A leap year has 366 days.
A common year has 365 days.
```

这样，你可以根据输入的参数来判断一年有多少天了。

一年有多少秒

```
#define SECONDS_IN_A_COMMON_YEAR (365 * 24 * 60 * 60)
#define SECONDS_IN_A_LEAP_YEAR (366 * 24 * 60 * 60)
```

示例：求圆面积和周长

```
// 定义计算圆的面积的宏
#define CIRCLE_AREA(radius) (3.14159 * (radius) * (radius))

// 定义计算圆的周长的宏
#define CIRCLE_CIRCUMFERENCE(radius) (2 * 3.14159 * (radius))
```

示例：返回小值

宏可以用来替换复杂的代码表达式，简化代码的编写。

```
#define MIN(a, b) ((a) < (b) ? (a) : (b))
int min = MIN(5, 10);
```

示例：宏与函数的比较

宏定义一个加法，也经常考！

```
#include <stdio.h>

// 定义一个宏
#define ADD(a, b) ((a) + (b))

// 定义一个函数
int add(int a, int b) {
    return a + b;
}

int main() {
    int result1 = ADD(3, 4);
    int result2 = add(3, 4);
    printf("Result of macro: %d\n", result1);
    printf("Result of function: %d\n", result2);
    return 0;
}
```

在这个示例中，我们定义了一个 `ADD` 宏和一个 `add` 函数，它们都用来计算两个数的和，并在 `main` 函数中调用。

这些示例展示了 `#define` 在不同场景下的使用方式，包括简单的宏定义、带参数的宏、字符串化宏以及宏与函数的比较。

!!!! 带参数的宏与函数的区别，面试会问，能回答以下三个就够用!!!! :

- 宏在预处理阶段展开，没有类型检查，而函数调用在编译时处理，有类型检查。
- 宏可以处理多行语句，而函数不能直接返回多行语句。

- 宏的参数没有存储空间的概念，而函数的参数在调用时会占用栈空间。

下面是带参数宏定义和函数的区别，以表格形式展示：了解一下就行，主要是上面三点

特性	带参数宏定义	函数
定义	使用 <code>#define</code> 和替换文本定义宏	使用 <code>return</code> 语句和花括号 <code>{}</code> 定义函数
类型检查	没有类型检查，仅文本替换	有类型检查，参数和返回值都有明确类型
作用域	宏在预处理阶段展开，没有实际的代码作用域概念	函数有作用域，可以访问局部变量和参数
调用方式	直接使用宏名和参数，如 <code>MACRO_NAME(arg1, arg2)</code>	调用函数名和参数，如 <code>functionName(arg1, arg2)</code>
存储	不占用内存，宏展开后替换为代码	函数体存储在内存中，占用空间
效率	通常比函数调用更快，因为它们在编译前展开，没有调用开销	函数调用有额外的调用和返回开销
调试	调试时看到的是展开后的代码，可能难以追踪原始宏调用	调试时可以轻松定位到函数调用和函数体
副作用	宏展开可能导致意外行为，如重复计算或与运算符优先级相关的问题	函数调用不会导致这类问题
多行语句	可以定义包含多行语句的宏	函数体可以包含多行语句
返回值	宏不返回值，它们只是代码替换	函数可以返回值
内存管理	不涉及内存分配和释放	函数可以使用动态内存分配（如 <code>malloc</code> 、 <code>free</code> ）
重入性	宏可能不是重入的，因为它们可能依赖全局状态或外部变量	函数设计为重入的，不依赖于外部状态

这个表格总结了带参数宏定义和函数在定义、类型检查、作用域、调用方式、存储、效率、调试、副作用、多行语句、返回值和内存管理等方面的区别。选择使用宏还是函数，取决于具体的应用场景和对上述特性的需求。

示例：字符串化运算符：

这个出现不多-但它是陷阱，也许在选择题中

字符串化运算符（`#`）是 C 语言预处理中的一个特性，它用于将宏的参数转换成一个字符串字面量。这个运算符在宏定义中非常有用，尤其是当你需要在编译时将某个标识符或表达式转换成字符串时。

```
#include <stdio.h>

// 这个宏 STRINGIFY 接受一个参数 x，并将其转换成一个字符串。当你使用这个宏时，x 被替换为 #x，
// 预处理器会将 x 视为一个字符串。
#define STRINGIFY(x) #x

int main() {
    const char* name = "Kimi";
    // 直接打印变量的值
    printf("The name is: %s\n", name);
    // 使用宏将变量名转换为字符串，并打印
    printf("The name of the variable is: %s\n", STRINGIFY(name));
    return 0;
}
```

当你编译并运行这段代码时，它会输出：

```
The name is: Kimi
The name of the variable is: name
```

所以这是个陷阱，我们以为像define一个PI一样，会输出3.14159，以为这里会输出Kimi，其实不是!!!

在这个示例中，STRINGIFY 宏将 name 转换成了字符串 "name"，并在 printf 函数中使用。

1.1.3 宏的取消定义 (#undef)

宏的取消定义（#undef）是一个预处理指令，用于取消之前定义的宏。这在某些情况下很有用，比如当你想要重新定义一个宏或者在条件编译中控制宏的定义。

以下是一个包含 main 函数的示例代码，展示如何使用 #undef 来取消宏的定义：

```
#include <stdio.h>

// 定义一个宏
#define CHEN "very handsome"

int main() {
    // 使用宏
    printf("%s\n", CHEN);

    // 取消宏的定义
    #undef CHEN

    // 尝试使用已取消定义的宏（这将导致编译错误）
    // printf("%s\n", GREETING);

    return 0;
}
```

在这个示例中，我们首先定义了一个名为 CHEN 的宏，它包含了一个字符串。在 main 函数中，我们使用这个宏来打印一条消息。接着，我们使用 #undef 指令取消了 CHEN 宏的定义。如果我们尝试再次使用 CHEN 宏，预处理器会报错，因为它已经不再定义了。

编译并运行这段代码时，输出将会是：


```
very handsome
```

1.1.4 修饰头文件

假设我们有一个头文件 `myheader.h`，我们希望确保其中的内容只被包含一次：

```
// myheader.h
#ifndef MYHEADER_H
#define MYHEADER_H

// 头文件内容
void myFunction() {
    // 函数实现
}

#endif // MYHEADER_H
```

在这个头文件中，`#ifndef MYHEADER_H` 检查 `MYHEADER_H` 宏是否未定义，如果是，则定义它，并包含头文件的内容。`#endif` 标记了条件编译块的结束。

现在，让我们看看如何在一个 C 程序中使用这个头文件：

```
#include <stdio.h>

// 假设这是我们的头文件
#ifndef MYHEADER_H
#define MYHEADER_H

void myFunction() {
    printf("Hello from myFunction!\n");
}

#endif // MYHEADER_H

int main() {
    myFunction(); // 调用头文件中定义的函数
    return 0;
}
```

在这个程序中，当我们包含 `myheader.h` 时，`#ifndef`、`#define` 和 `#endif` 确保了即使 `myheader.h` 被多次包含（例如，通过多个不同的源文件），`myFunction` 也只会定义一次。这样可以避免链接错误和重复定义的问题。

编译并运行这段代码时，输出将会是：

```
Hello from myFunction!
```

这种使用 `#ifndef`、`#define` 和 `#endif` 的技术被称为“包含卫士”或“头文件卫士”，是管理头文件内容被多次包含的一种标准方法。

1.2 typedef关键字-别名作用

`typedef` 是 C 语言中的一个关键字，用于为数据类型定义一个新的名称，称为别名（alias）。使用 `typedef` 可以增加代码的可读性，使得类型定义更加清晰和简洁。

1.2.1 为基本数据类型定义别名

```
#include <stdio.h>

// 为基本数据类型定义别名
typedef unsigned char Byte;      // 8位无符号整数
typedef char* String;           // 字符串类型
typedef float Real;              // 单精度浮点数
typedef int Bool;                // 布尔类型，用整型表示
typedef unsigned long ulong;    // 无符号长整数

// 为布尔类型定义两个常量
#define TRUE 1
#define FALSE 0

int main() {
    // 使用别名声明变量
    Byte byteValue = 0x0F;
    String str = "Hello, world!";
    Real realValue = 3.14159f;
    Bool boolValue = TRUE;
    ulong ulongValue = 1234567890UL;

    // 打印变量值
    printf("Byte value: %u\n", byteValue);
    printf("String: %s\n", str);
    printf("Real value: %f\n", realValue);
    printf("Boolean value: %s\n", boolValue ? "TRUE" : "FALSE");
    printf("Unsigned long value: %lu\n", ulongValue);

    return 0;
}
```

1.2.2 为复杂的数据类型定义别名

以下是使用 `typedef` 为复杂数据类型（如结构体和联合体）定义别名，并在 `main` 函数中使用这些别名的示例代码：

```
#include <stdio.h>

// 为结构体定义别名
typedef struct {
    int id;
    float salary;
    char name[50];
} Employee;

// 为联合体定义别名
```

```

typedef union {
    int intValue;
    float floatValue;
    char charValue;
} Data;

int main() {
    // 使用结构体别名声明变量
    Employee emp = {1, 50000.0f, "John Doe"};

    // 使用联合体别名声明变量
    Data data;
    data.intValue = 10;

    // 打印结构体变量的值
    printf("Employee ID: %d\n", emp.id);
    printf("Employee Salary: %.2f\n", emp.salary);
    printf("Employee Name: %s\n", emp.name);

    // 打印联合体变量的值
    printf("Data as int: %d\n", data.intValue);

    // 重新赋值并打印联合体变量的值
    data.floatValue = 3.14f;
    printf("Data as float: %.2f\n", data.floatValue);

    // 注意：由于联合体的特性，打印 intValue 可能会得到意外的结果
    // printf("Data as int after float assignment: %d\n", data.intValue);

    return 0;
}

```

之前我们可能是这么写的

```

#include <stdio.h>

// 为结构体定义别名
struct Person{
    int id;
    float salary;
    char name[50];
};

int main() {
    struct Person P1 = {1, 50000.0f, "John Doe"};;
    struct Person P2; //在定义的时候要加struct关键字

    // 使用结构体别名声明变量,就不用加struct关键字了
    // Employee emp = {1, 50000.0f, "John Doe"};
    return 0;
}

```

1.2.3 为数组类型定义别名

这个用的不多，有点怪，可以简单涉猎下

这里有一个使用 `typedef` 为数组类型定义别名，并在 `main` 函数中使用这些别名的示例代码：

```
#include <stdio.h>

// 为数组类型定义别名
typedef char CharArray[100]; // 固定大小的字符数组
typedef int IntArray[10];    // 固定大小的整数数组

int main() {
    // 使用别名声明数组变量
    CharArray name = "Kimi";
    IntArray scores;

    // 初始化数组
    for (int i = 0; i < 10; ++i) {
        scores[i] = i * 10;
    }

    // 打印字符数组
    printf("CharArray: %s\n", name);

    // 打印整数数组
    printf("IntArray contains: ");
    for (int i = 0; i < 10; ++i) {
        printf("%d ", scores[i]);
    }
    printf("\n");

    return 0;
}
```

在这个示例中，我们定义了两个数组类型的别名：

- `CharArray` 作为包含100个字符的数组的别名。
- `IntArray` 作为包含10个整数的数组的别名。

在 `main` 函数中，我们使用这些别名来声明 `CharArray` 类型的变量 `name` 和 `IntArray` 类型的变量 `scores`。然后，我们为 `scores` 数组中的每个元素赋值，并使用 `printf` 函数打印 `name` 和 `scores` 数组的内容。

这个示例展示了如何使用 `typedef` 为数组类型创建更简洁的别名，使得数组的声明更加方便和清晰。

1.2.4 为指针类型定义别名

以下是使用 `typedef` 为指针类型定义别名，并在 `main` 函数中使用这些别名的示例代码：

```
#include <stdio.h>
#include <stdlib.h>

// 为指针类型定义别名
```

```

typedef char* PChar;      // 指向字符的指针
typedef int* PInt;        // 指向整数的指针
typedef float* PFloat;    // 指向浮点数的指针

int main() {
    // 使用别名声明指针变量
    PChar str = malloc(100 * sizeof(char)); // 分配内存并初始化字符串
    PInt num = malloc(sizeof(int));          // 分配内存以存储整数
    PFloat value = malloc(sizeof(float));    // 分配内存以存储浮点数

    // 初始化指针指向的值
    *str = 'H'; // C风格字符串以'\0'结尾，这里仅为示例，不进行完整字符串赋值
    *str = '\0'; // 确保字符串正确终止
    *num = 42;
    *value = 3.14f;

    // 打印指针指向的值
    printf("Pointer to char points to: %s\n", str);
    printf("Pointer to int points to: %d\n", *num);
    printf("Pointer to float points to: %.2f\n", *value);

    // 释放内存
    free(str);
    free(num);
    free(value);

    return 0;
}

```

1.2.5 为函数指针类型定义别名

为函数指针类型定义别名时，我们使用 `typedef` 关键字来创建一个新的类型名，这样可以使得函数指针类型的声明更加简洁和清晰。以下是一个示例代码，展示如何为函数指针类型定义别名，并在 `main` 函数中使用这个别名：

```

#include <stdio.h>

// 为函数指针类型定义别名
// 这个函数指针接受两个int参数并返回一个int
typedef int (*IntFunctionPtr)(int, int);

int add(int a, int b) {
    return a + b;
}

int subtract(int a, int b) {
    return a - b;
}

int main() {
    // 使用别名声明函数指针变量
    IntFunctionPtr funcPtr;

    // 给函数指针变量赋值
    funcPtr = add; // 指向add函数
}

```

```

    printf("10 + 5 = %d\n", funcPtr(10, 5));

    funcPtr = subtract; // 指向subtract函数
    printf("10 - 5 = %d\n", funcPtr(10, 5));

    return 0;
}

```

在这个示例中，我们定义了一个名为 `IntFunctionPtr` 的别名，它是一个函数指针类型，指向接受两个 `int` 参数并返回一个 `int` 的函数。然后我们定义了两个简单的函数 `add` 和 `subtract`，它们都符合这个签名。

在 `main` 函数中，我们声明了一个 `IntFunctionPtr` 类型的变量 `funcPtr`，并分别将其指向 `add` 和 `subtract` 函数。然后通过函数指针调用这些函数，并打印结果。

这个示例展示了如何使用 `typedef` 为函数指针类型定义别名，这样做可以使得代码更加简洁，特别是在处理复杂的函数指针时。使用别名可以避免每次都写出完整的函数指针类型，从而使代码更加易读。

以下是函数指针不用别名的方式，回忆一下：

```

#include <stdio.h>

// 定义两个符合特定签名的函数
int add(int a, int b) {
    return a + b;
}

int subtract(int a, int b) {
    return a - b;
}

int main() {
    // 声明函数指针变量，但不使用别名
    int (*funcPtr)(int, int);

    // 给函数指针变量赋值
    funcPtr = add; // 指向add函数
    printf("10 + 5 = %d\n", funcPtr(10, 5));

    funcPtr = subtract; // 指向subtract函数
    printf("10 - 5 = %d\n", funcPtr(10, 5));

    return 0;
}

```

1.2.6 特点总结

- `typedef` 创建的是真实的类型别名，它不仅用于代码的阅读，而且在编译时也会将别名替换为实际的类型。
- `typedef` 定义的别名在编译时会进行类型检查，这是它与宏定义的主要区别之一。
- `typedef` 可以减少代码中的复杂性，特别是在处理嵌套的数据结构时。
- `typedef` 可以提高代码的可移植性，因为你可以为特定平台定义特定的类型别名。

示例代码

以下是一个包含 `main` 函数的示例代码，展示如何使用 `typedef`：

```
#include <stdio.h>

// 使用 typedef 为 int 定义别名
typedef int Length;

// 使用 typedef 为结构体定义别名
typedef struct {
    Length length;
    Length width;
} Rectangle;

int main() {
    Rectangle rect;
    rect.length = 10;
    rect.width = 5;

    // 使用 typedef 定义的别名
    Length perimeter = 2 * (rect.length + rect.width);
    printf("The perimeter of the rectangle is: %d\n", perimeter);
    return 0;
}
```

在这个示例中，我们为 `int` 类型定义了一个别名 `Length`，并为一个结构体定义了一个别名 `Rectangle`。在 `main` 函数中，我们使用这些别名来声明变量和计算矩形的周长。

使用 `typedef` 可以让代码更加简洁和易于理解，尤其是在处理复杂的数据结构时。它也有助于在不同的平台之间保持代码的一致性，因为你可以为特定的数据类型定义统一的别名。

1.3 extern关键字

C语言中的 `extern` 关键字用于**声明**一个变量或函数是在另一个文件或编译单元中**定义**的。

<!! 什么是声明，什么是定义，本节后面讲，面试也会作怪问这种问题!! >

优点：或者说为什么要用extern，和头文件的区别是什么：

`extern` 关键字的使用可以减少代码重复，提高模块化，使得程序的各个部分可以更容易地共享资源。

以下是 `extern` 关键字的一些关键点：

1.3.1 全局变量的外部链接

当一个全局变量被声明为 `extern` 时，它允许当前文件访问在其他文件中定义的全局变量。

```
// file1.c
int global_var = 10;

// file2.c 当前文件file2.c访问其他文件file1.c中定义的全局变量
extern int global_var; // 声明global_var在其他文件定义
printf("%d", global_var); // 可以访问file1.c中定义的global_var，这里输出的结果是file1.c
中的10
```

1.3.2 函数的外部链接

类似地，`extern` 也用于声明函数，允许当前文件调用在其他文件中定义的函数。

```
// file1.c
#include <stdio.h>
void function() {
    // 函数实现
    printf("c1c handsome\n");
}

// file2.c
extern void function(); // 声明function在其他文件定义
function(); // 调用file1.c中定义的function,c1c handsome
```

1.3.3 头文件中的使用

`extern` 经常在头文件中使用，以声明全局变量和函数，这样多个文件就可以包含同一个头文件并访问这些全局资源，而不需要在每个文件中都重新定义它们。

```
// header.h
extern int global_var; // 声明变量
extern void function(); // 声明函数

// file1.c
#include "header.h"
int global_var = 10; // 定义变量
void function() {    // 定义并实现函数

}

// file2.c
#include "header.h"
function(); // 调用函数
```

默认的外部链接，上述案例中，不用`extern`也是可以的，因为

在C语言中，如果没有特别指定存储类，全局变量和全局函数默认具有外部链接。使用 `extern` 关键字可以明确指出这一点，尤其是在跨文件共享全局资源时。

1.3.4 声明和定义的区别

在编程语言中，特别是在C和C++中，"声明" (Declaration) 和"定义" (Definition) 是两个不同的概念，它们在代码中扮演着不同的角色：

声明 (Declaration) :

- 声明变量时，它不会分配内存空间；
- 声明函数时，它提供了函数的名称、返回类型和参数列表，但不包含函数体。
- 声明可以多次出现在程序中
- 声明通常用于头文件中，通过 `extern` 关键字来实现跨文件的引用。

例如：


```
int globalVar; // 变量声明
void function(); // 函数声明
```

定义 (Definition) :

- 定义变量时，它在内存中分配空间，并可能初始化。
- 定义函数时，它包含了函数的实现（函数体）。
- 每个标识符在程序中只能有一个定义。
- 定义通常出现在源文件（.c 或 .cpp 文件）中。

例如：

```
int globalVar = 42; // 变量定义
void function() {
    // 函数体
} // 函数定义
```

区别:

- **作用域:** 声明可以出现在头文件中，供多个源文件包含，而定义通常只出现在一个源文件中。
- **内存分配:** 声明不分配内存，定义变量时分配内存。
- **实现细节:** 声明不包含实现细节，定义包含实现细节，如函数体。
- **数量限制:** 一个标识符可以有多个声明，但只能有一次定义。
- **编译器行为:** 声明让编译器知道标识符的存在和类型，定义则让编译器生成相应的代码。

理解声明和定义的区别对于编写模块化和可重用的代码非常重要，尤其是在大型项目中，它有助于避免命名冲突并确保代码的正确链接。

1.4. static关键字

1.4.1 修饰局部变量

在函数内部声明的静态变量具有以下特性：

- **生命周期:** 静态变量的生命周期从程序开始执行到程序结束，它们在整个程序执行期间都存在，而不是在函数调用期间创建和销毁。
- **可见性:** 静态变量的作用域仅限于包含它们的函数内部，即它们是局部变量，但其值在函数调用之间保持不变。
- **初次初始化:** 静态变量只在第一次进入其定义的函数时进行初始化，并且仅执行一次。

```
#include <stdio.h>
void test()
{
    static int m = 0;
    m = m + 1;
    printf("%d", m);
}
```

```
int main()
{
    int n = 0;
    while (n < 10)
    {
        test();
        n++;
    }
}
```

输出结果：

```
12345678910
```

1.4.2 修饰全局变量

在函数外部（全局范围）声明的静态变量，又叫**全局静态变量**具有以下特性：

- 生命周期：静态全局变量的生命周期与程序的执行周期相同。
- 可见性：静态全局变量的作用域限于声明它们的源文件，不能被其他源文件使用。

创建一个源文件：a.c

```
int g_val=2024;//全局变量
```

创建一个源文件b.c，引用全局变量：全局变量的作用域是整个工程

```
#include <stdio.h>
extern int g_val;//extern 用来声明外部命令，可以调用到a.c的 g_val=2024
int main()
{
    printf("%d\n", g_val);
    return 0;
}
//两个文件一起编译：gcc a.c b.c ， 程序输出结果是2024
```

static可以修饰全局变量，在a.c 加上static看一下有什么不同：

```
static int g_val = 2024;//全局变量
```

两个文件一起编译：gcc a.c b.c

加上之后,程序编译报错, 被static修饰的全局变量可见性只在声明它的源文件，其他源文件无法使用。

报错：

```
G:\10月份技术方案>gcc a.c b.c
C:\Users\asus\AppData\Local\Temp\ccpByz5S.o:b.c:(.rdata$.refptr.g_val[.refptr.g_val]+0x0): undefined reference to `g_val'
collect2.exe: error: ld returned 1 exit status
```

```
C:\Users\asus\AppData\Local\Temp\ccpByz5S.o:b.c:
(.ldata$.refptr.g_val[.refptr.g_val]+0x0): undefined reference to `g_val'
collect2.exe: error: ld returned 1 exit status
```

1.4.3 static修饰函数

使用'static'关键字声明的函数是静态函数。静态函数具有以下特性：

- 可见性：静态函数的作用域限于声明它们的源文件，不能被其他源文件调用。
- 隐藏性：将函数声明为静态的，可以将其隐藏在当前源文件中，以防止与其他源文件中具有相同名称的函数发生冲突。

源文件a.c中创建个函数名为add：

代码如下：

```
static int add(int x, int y)
{
    int z = x + y;
    return z;
}
```

主文件b.c如下：

```
#include <stdio.h>

extern add(int x, int y);
int main()
{
    int a = 2;
    int b = 3;
    int c = add(a, b);
    printf("%d\n", c);
    return 0;
}
```

其实一个函数本身具有外部链接属性 被static修饰后外部链接属性变成了内部链接属性，只能在源文件a.c内部使用了，其他源文件无法使用，使用上感觉作用域变小，我们可以发现 static 修饰全局变量和修饰函数用法一样。

编译后发现：

```
G:\10月份技术方案>gcc a.c b.c
C:\Users\asus\AppData\Local\Temp\ccqscgeS.o:b.c:(.text+0x24): undefined reference to `add'
collect2.exe: error: ld returned 1 exit status
```

```
C:\Users\asus\AppData\Local\Temp\ccqscgeS.o:b.c:(.text+0x24): undefined reference
to `add'
collect2.exe: error: ld returned 1 exit status
```

1.4.4 static修饰的变量存放在哪里

能回答的上这个，面试好感上升，正常面试者只会零散回答前面3个

static修饰的变量存放在静态数据区。



在C语言中，static变量

static修饰的变量	bss段	数据段
未显式初始化的static局部变量	bss段	
未显式初始化的static全局变量	bss段	
初始化为0的static局部变量	bss段	
初始化为0的static全局变量	bss段	
显式初始化的static全局变量		数据段

static修饰的变量	bss段	数据段
显式初始化的static局部变量	bss段 显式初始化的 <code>static</code> 局部变量实际上存储在BSS段，因为即使它们被显式初始化，它们的存储位置并没有改变。然而，它们的初始化时机和行为与其他BSS段变量不同。<code>static</code> 局部变量的初始化发生在第一次进入包含它们的函数时，而不是在main函数执行之前。此外，它们的生命周期持续到程序结束，而不是在函数调用结束后释放。 《请注意，<code>static</code> 局部变量的存储位置在不同编译器或不同优化级别下可能会有所不同，但它们通常在BSS段，并且初始化时机是在第一次函数调用时。》	

在程序启动时，BSS段会自动被操作系统或加载器初始化为零值。

顺便说说对象的存储，可分为三类：静态存储（static storage）；自动存储（automatic storage）；动态分配存储（allocated or dynamic storage）。

对于自动存储则对应的是栈（stack），动态分配存储对应的是堆（heap）；静态存储可分为.bss/.data/.rodata等数据段（section）。

在程序执行中把初始值为零或者是未设初始值的变量放在.bss段中。

1.5 const关键字

`const` 关键字在 C 语言中有多个作用，主要用于定义常量

保证变量的值在初始化后不可更改，说白了，变量的值不能被修改，就是常量了。

总的来说，`const`关键字在C语言中提供了一种机制来声明只读变量和指针，从而提高了代码的可读性、可维护性和安全性。

通过正确地理解和使用`const`，你可以确保某些数据在程序运行时不会被意外修改。

以下是 `const` 关键字的一些主要作用和相应的示例代码：

1.5.1 定义常量

```
#include <stdio.h>

int main() {
    const int MAX_USERS = 100; // 定义常量，值不可更改
    printf("Maximum number of users: %d\n", MAX_USERS);
    return 0;
}
```

在这个示例中，`MAX_USERS` 被定义为一个常量，其值在程序运行期间不能被修改。

1.5.2 修饰函数参数

```
#include <stdio.h>

void printMessage(const char* message) {
    // 函数内部不能修改 message 的值
    printf("%s\n", message);
}

int main() {
    const char* greeting = "Hello, world!";
    printMessage(greeting);
    return 0;
}
```

在这个示例中，`printMessage` 函数接受一个 `const` 参数，表明在函数内部不能修改 `message` 的值。

1.5.3 修饰函数返回值

```
#include <stdio.h>

const char* getAppName() {
    return "MyApp"; // 返回一个指向常量字符串的指针
}

int main() {
    const char* appName = getAppName();
    printf("Application Name: %s\n", appName);
    return 0;
}
```

在这个示例中，`getAppName` 函数返回一个指向常量字符串的指针，这意味着通过 `appName` 指针不能修改字符串的内容。

1.5.4 修饰指针

```
#include <stdio.h>

int main() {
    const int num = 42;
    int const *ptr = &num; // 指向常量的指针
    printf("%d\n", *ptr);
    // *ptr = 10; // 错误：不能通过 ptr 修改 num 的值
    return 0;
}
```

在这个示例中，`ptr` 是一个指向常量的指针，意味着不能通过 `ptr` 来修改它所指向的值。

1.5.5 修饰数组

```
#include <stdio.h>

int main() {
    const int numbers[] = {1, 2, 3, 4, 5};
    int length = sizeof(numbers) / sizeof(numbers[0]);
    for (int i = 0; i < length; i++) {
        printf("%d ", numbers[i]);
    }
    printf("\n");
    return 0;
}
```

在这个示例中，`numbers` 是一个常量数组，意味着数组中的元素在程序运行期间不能被修改。

`const` 关键字的使用可以提高程序的安全性和可读性，它告诉编译器和程序员哪些变量是不应该被修改的。

1.6 sizeof关键字

`sizeof` 是 C 语言中的一个编译时运算符，注意哦，不是函数，像函数，不是函数！！面试有可能会问。

用于确定变量或类型在内存中的大小（以字节为单位）。`sizeof` 可以用于基本数据类型、复合数据类型（如结构体和联合体）以及指针类型。

以下是 `sizeof` 的一些常见用法和特点：

1.6.1 基本用法

1. 获取基本数据类型的大小：

```
printf("Size of char: %zu\n", sizeof(char));
printf("Size of int: %zu\n", sizeof(int));
printf("Size of float: %zu\n", sizeof(float));
```

2. 获取复合数据类型的大小：

```
struct MyStruct {
    int a;
    float b;
    char c;
};

printf("Size of MyStruct: %zu\n", sizeof(struct MyStruct));
```

3. 获取指针类型的大小：

```
int var = 10;
printf("Size of int pointer: %zu\n", sizeof(&var));
```

1.6.2 特点

- `sizeof` 返回的是类型的存储大小，不是类型中元素的数量。
- `sizeof` 可以用于任何数据类型，包括原始类型（如 `int`、`float`）和复合类型（如 `struct`、`union`）。
- `sizeof` 的结果是一个常量，因此可以在编译时计算。
- `sizeof` 可以用于函数参数，但不能用在运行时的变量或表达式上。

1.6.3 示例代码

以下是一个包含 `main` 函数的示例代码，展示如何使用 `sizeof`：

```
#include <stdio.h>

int main() {
    int intVar = 10;
    float floatVar = 3.14f;
    double doubleVar = 3.14159;
    char charVar = 'A';
    struct {
        int a;
        float b;
        char c;
    } structVar;

    printf("Size of int: %zu\n", sizeof(intVar));
    printf("Size of float: %zu\n", sizeof(floatVar));
    printf("Size of double: %zu\n", sizeof(doubleVar));
    printf("Size of char: %zu\n", sizeof(charVar));
    printf("Size of struct: %zu\n", sizeof(structVar));

    return 0;
}
```

在这个示例中，我们声明了几种不同类型的变量，并使用 `sizeof` 运算符来获取它们的大小。`%zu` 是 `printf` 的格式说明符，用于打印 `size_t` 类型的值，这是 `sizeof` 运算符的结果类型。

`sizeof` 是一个非常有用的工具，可以帮助你了解数据类型在内存中的布局，对于内存管理、数组操作和数据对齐等场景非常有用。

1.6.4 `sizeof`和`strlen`区别

很重要，新手非常非常非常非常非常经常犯下的错误，特别是编程过程中

这段代码提供了一个很好的例子来说明 `sizeof` 和 `strlen` 之间的区别。让我们来分析这段代码并分析输出。

特别注意 `str2[100]`!!!

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "Hello, world!";
    char str2[100] = "Hello, world!";
```



```

char *ptr = "Hello";

// 使用 sizeof 获取类型大小
printf("Size of int: %zu\n", sizeof(int));
printf("Size of str: %zu\n", sizeof(str)); // 注意: sizeof 得到的是整个数组的大小
printf("Size of str2: %zu\n", sizeof(str2)); // 注意: sizeof 得到的是整个数组的大
小

// printf("Length of str2: %zu\n", strlen(str2)); // 特别注意这里的 strlen(str2) 是
多少! !!! ?
printf("Size of ptr: %zu\n", sizeof(ptr)); // sizeof 得到的是指针的大小, 不是字符
串的长度

// 使用 strlen 获取字符串长度
printf("Length of str: %zu\n", strlen(str));
printf("Length of str2: %zu\n", strlen(str2));
printf("Length of ptr: %zu\n", strlen(ptr));

return 0;
}

```

输出

Size of int: 这将输出 `int` 类型的大小, 通常在大多数系统上是 4 字节。

```
Size of int: 4
```

Size of str: 这将输出数组 `str` 的大小。 `str` 包含字符串 `"Hello, world!"`, 后面还有一个隐含的空字符 `\0`, 共 14 字节。

```
Size of str: 14
```

Size of str2: 这将输出数组 `str2` 的大小, 它被显式地声明为大小为 100 的数组。

```
Size of str2: 100
```

Size of ptr: 这将输出指针 `ptr` 的大小, 而不是它指向的字符串的长度。在 32 位系统上, 指针通常是 4 字节; 在 64 位系统上, 通常是 8 字节。

```
Size of ptr: 8 (在 64 位系统上)
```

Length of str: 这将输出字符串 `str` 的长度, 不包括结尾的空字符 `\0`。字符串 `"Hello, world!"` 的长度是 13。

```
Length of str: 13
```

Length of ptr: 这将输出字符串 `ptr` 指向的 `"Hello"` 的长度, 不包括结尾的空字符 `\0`。字符串 `"Hello"` 的长度是 5。

```
Length of ptr: 5
```

区别阐述

- `sizeof`: 用于获取变量或类型在内存中的大小。它是一个编译时运算符，可以用于任何数据类型，包括数组和指针。`sizeof` 对于数组，返回整个数组的大小；对于指针，返回指针本身的大小，而不是指针指向的内容的大小。
- `strlen`: 用于计算以空字符 `\0` 结尾的字符串的长度。它是一个运行时函数，需要一个指向字符数组的指针，并返回字符串的长度，不包括结尾的空字符。`strlen` 只能用于以 `\0` 结尾的字符串，不能用于一般的字符数组。

通过这个代码示例，我们可以清楚地看到 `sizeof` 和 `strlen` 的不同用途和行为。`sizeof` 提供了关于数据类型大小的信息，而 `strlen` 提供了关于字符串内容长度的信息。

1.7 register关键字

不多见，可以了解一下!

在 C 语言中，`register` 关键字是一个建议性的指示符，用于建议编译器将变量存储在 CPU 的寄存器中，而不是 RAM 中。使用 `register` 关键字的目的是为了优化程序性能，因为访问寄存器比访问内存要快得多。

1.7.1 基本用法

```
register int count;
```

在这个例子中，`count` 被声明为一个 `register` 类型的整型变量，这意味着编译器被建议将这个变量存储在寄存器中。

1.7.2 示例代码

以下是一个包含 `main` 函数的示例代码，展示如何使用 `register` 关键字：

```
#include <stdio.h>

int main() {
    register int i; // 建议编译器将 i 存储在寄存器中

    for (i = 0; i < 10; i++) {
        printf("%d ", i);
    }
    printf("\n");

    return 0;
}
```

在这个示例中，变量 `i` 被声明为 `register` 类型，建议编译器在循环中使用寄存器来存储 `i` 的值，以提高循环的执行速度。

尽管 `register` 关键字在某些情况下可以提供性能优化，但在现代编程实践中，它的使用越来越少，因为编译器的自动优化通常更有效。此外，`register` 的使用可能会限制变量的可见性和调试能力。

1.8 volatile关键字

重要，经常考！

以下那段话，可以背诵或者阐述大概意思，面试会问

`volatile` 是 C 语言中的一个关键字，用于告知编译器该变量可能被程序之外的因素（例如硬件或其他线程）以不可预测的方式改变。使用 `volatile` 关键字的变量，编译器将不会对该变量进行优化，每次访问这个变量时都会从它的存储地址中重新读取值，而不是使用寄存器中的值或缓存的值。

1.8.1 基本用法

```
volatile int flag;
```

在这个例子中，`flag` 被声明为一个 `volatile` 变量，这意味着每次对 `flag` 的访问都应该是直接从它的地址读取，而不是从寄存器或缓存中读取。

1.8.2 示例代码

以下是一个包含 `main` 函数的示例代码，展示如何使用 `volatile` 关键字：

```
#include <stdio.h>

// 假设这是由硬件或其他线程修改的全局变量
volatile int globalFlag = 0;

void interruptServiceRoutine() {
    // 模拟硬件中断，改变全局变量的值
    globalFlag = 1;
}

int main() {
    // 模拟等待中断的过程
    while (globalFlag == 0) {
        // 循环检查全局标志是否被设置
    }

    // 一旦标志被设置，执行相应的操作
    printf("Interrupt service routine has been called.\n");

    return 0;
}
```

在这个示例中，`globalFlag` 是一个 `volatile` 变量，它可能被中断服务程序 `interruptServiceRoutine` 改变。在 `main` 函数中，程序循环检查 `globalFlag` 的值，一旦它被设置，就执行相应的操作。

使用 `volatile` 关键字是处理硬件寄存器或多线程环境中变量时的一个重要机制，它确保了程序能够正确响应外部变化。然而，`volatile` 不应该被滥用，因为它不会提供线程安全或同步保证。在需要同步多个线程对共享资源的访问时，应该使用适当的同步机制，如互斥锁（mutexes）或原子操作。

1.9 enum关键字

`enum` 是 C 语言中的一个关键字，用于定义枚举类型，这是一种用户定义的类型，它允许为整型常量赋予有意义的名称。枚举是一种强类型的数据类型，可以提高代码的可读性和可维护性。

1.9.1 基本用法

1. 定义枚举类型：

```
enum Color { RED, GREEN, BLUE };
```

这里定义了一个名为 `Color` 的枚举类型，它包含三个枚举常量：`RED`、`GREEN` 和 `BLUE`。

2. 定义枚举类型并指定底层类型：

```
enum Color : unsigned int { RED = 10, GREEN = 20, BLUE = 30 };
```

在这个例子中，枚举 `Color` 被指定了底层类型 `unsigned int`，并且为每个枚举常量显式赋值。

1.9.2 特点和注意事项

- 默认值：**如果枚举常量没有显式赋值，它们将从 `0` 开始自动赋值，每个后面的枚举常量值比前一个值大 `1`。
- 底层类型：**枚举类型有一个底层类型，通常是 `int`，但也可以显式指定为其他整型类型，如 `unsigned int`、`long` 等。
- 存储大小：**枚举类型变量的存储大小取决于其底层类型。
- 类型安全：**枚举提供了类型安全的方式，限制变量只能取特定的几个值之一。
- 跨平台兼容性：**枚举常量在不同平台上的底层表示可能不同，因此不应依赖于特定的底层值。

1.9.3 示例代码

以下是一个包含 `main` 函数的示例代码，展示如何使用 `enum` 关键字：

```
#include <stdio.h>

// 定义一个枚举类型表示星期几
enum Weekday { SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY };

int main() {
    enum Weekday today = TUESDAY;
    printf("Today is %d\n", today);

    switch (today) {
        case SUNDAY:
            printf("Sunday is the first day of the week.\n");
            break;
        case SATURDAY:
            printf("Saturday is the last day of the week.\n");
            break;
        default:
            printf("Today is a weekday.\n");
    }

    return 0;
}
```

```
}
```

```
1  #include <stdio.h>
2
3  // 定义一个枚举类型表示星期几
4  enum Weekday { SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY,
5
6  int main() {
7      enum Weekday today = TUESDAY;
8      printf("Today is %d\n", today);
9
10     switch (today) {
11         case SUNDAY:
12             printf("Sunday is the first day of the week.\n");
13             break;
14         case SATURDAY:
15             printf("Saturday is the last day of the week.\n");
16             break;
17         default:
18             printf("Today is a weekday.\n");
19     }
20
21     return 0;
22 }
```

管理员: 命令提示符

G:\10月份技术方案>gcc a.c
G:\10月份技术方案>a.exe
Today is 2
Today is a weekday.
G:\10月份技术方案>
运行结果

在这个示例中，我们定义了一个名为 `Weekday` 的枚举类型来表示一周的七天。在 `main` 函数中，我们声明了一个 `Weekday` 类型的变量 `today` 并给它赋值为 `TUESDAY`，然后使用 `switch` 语句根据 `today` 的值打印相应的信息。

使用 `enum` 可以使得代码更加清晰和易于理解，尤其是在处理一组有限的、相关的常量时。

1.10 auto关键字

这部分简单了解一下。

在 C 语言的历史中，`auto` 关键字最初用于声明自动存储周期的局部变量。然而，在 C99 标准之后，`auto` 的这一用法变得不必要，因为自动存储周期成了局部变量的默认行为。因此，在现代 C 语言编程中，`auto` 关键字的使用越来越少，它通常不被用于声明变量。

1.10.1 基本用法（历史用法）

在 C89/C90 标准中，`auto` 关键字用于声明具有自动存储周期的局部变量，这意味着变量的生命周期仅限于其定义的代码块，并且在函数调用结束后会被销毁。

```
auto int localVar = 10; // 声明一个自动存储周期的整型变量
```

1.10.2 现代用法

在 C99 及以后的版本中，所有的局部变量默认具有自动存储周期，因此 `auto` 关键字变得多余，可以省略。

```
int localVar = 10; // auto 可以省略，这是默认行为
```

1.10.3 C++ 中的 `auto` 关键字

在 C++ 中，`auto` 关键字有一个更现代的用法，它用于自动类型推断。编译器会根据变量的初始值自动确定其类型。

```
auto x = 10;           // x 的类型被推断为 int
auto y = 3.14;         // y 的类型被推断为 double
auto z = "hello";      // z 的类型被推断为 const char*
```

在 C++ 中，`auto` 关键字非常有用，特别是在处理复杂的数据类型或泛型编程时，它可以简化代码并减少打字量。

1.10.4 C11 中的 `auto` 关键字

在 C11 标准中，`auto` 关键字与 `sizeof` 运算符一起使用时，可以用于避免类型转换。具体来说，当你有一个复杂的类型，比如一个数组或者一个结构体，并且你想要获取这个类型的一个元素或者成员的大小时，你可以使用 `auto` 来简化这个过程。

以下是包含 `main` 函数的示例代码，展示如何使用 `auto` 与 `sizeof`：

```
#include <stdio.h>

int main() {
    // 假设有一个结构体
    struct myStruct{
        char a;
        int b;
        double c;
    };

    // 使用 auto 和 sizeof 获取结构体成员的大小
    auto sizeofChar = sizeof(myStruct.a);
    auto sizeofInt = sizeof(myStruct.b);
    auto sizeofDouble = sizeof(myStruct.c);

    // 打印结果
    printf("Size of char: %zu\n", sizeofChar);
    printf("Size of int: %zu\n", sizeofInt);
    printf("Size of double: %zu\n", sizeofDouble);

    return 0;
}

//关于%zu的说明
/*
```

`%zu` 告诉 `printf` 函数期待一个 `size_t` 类型的参数，并且按照适当的方式格式化输出这个值。如果不使用 `%zu` 而使用 `%d` 或 `%u`，那么在 64 位系统上可能无法正确显示大于 4GB 的数组大小，因为 `%d` 和 `%u` 分别对应 `int` 和 `unsigned int` 类型，它们可能不足以存储 `size_t` 类型的所有可能值。

*/

在这个示例中，我们定义了一个结构体 `myStruct`，并且使用 `auto` 与 `sizeof` 运算符来获取结构体中每个成员的大小。`auto` 在这里用于自动推断 `sizeof` 运算符的结果类型，即 `size_t` 类型，这样就避免了显式地写出 `size_t` 类型。

这种用法在处理复杂的数据结构或者数组时特别有用，因为它可以减少代码的复杂性，并且避免类型转换的错误。通过使用 `auto`，编译器会自动处理类型转换，使得代码更加简洁和安全。

总的来说，`auto` 关键字在现代 C 语言中的使用较少，而在 C++ 中则更为常见，用于自动类型推断。在 C11 中，`auto` 用于简化与 `sizeof` 相关的声明。

1.11 C语言预处理

C语言预处理（C Preprocessing）是指在C语言编译过程中的一个阶段，发生在实际编译之前。预处理器（preprocessor）负责处理源代码文件中的预处理指令。这些指令不是C语言的一部分，而是预处理器的命令，它们以井号（#）开头。

1.11.1 #include

```
#include <stdio.h>

int main() {
    printf("Hello, world!\n");
    return 0;
}
```

用法：`#include` 指令用于包含标准库头文件或用户自定义头文件。在这个例子中，我们包含了标准输入输出库 `stdio.h`，这样我们就可以在程序中使用 `printf` 函数。

输出： Hello, world!

1.11.2 #define

```
#include <stdio.h>

#define GREET "Hello, world!"

int main() {
    printf("%s\n", GREET);
    return 0;
}
```

用法：`#define` 指令用于定义宏。这里我们定义了一个宏 `GREET`，它的值是一个字符串。

输出： Hello, world!

1.11.3 #undef

```
#include <stdio.h>

#define MESSAGE "Hello, world!"
#undef MESSAGE

int main() {
    // MESSAGE is now undefined
    return 0;
}
```

用法：`#undef` 指令用于取消宏的定义。在这个例子中，`MESSAGE` 宏首先被定义，然后被取消定义。

输出：编译错误，因为 `MESSAGE` 在 `main` 函数中被取消定义，无法使用。

1.11.4 #ifdef, #ifndef, #if, #else, #elif, #endif

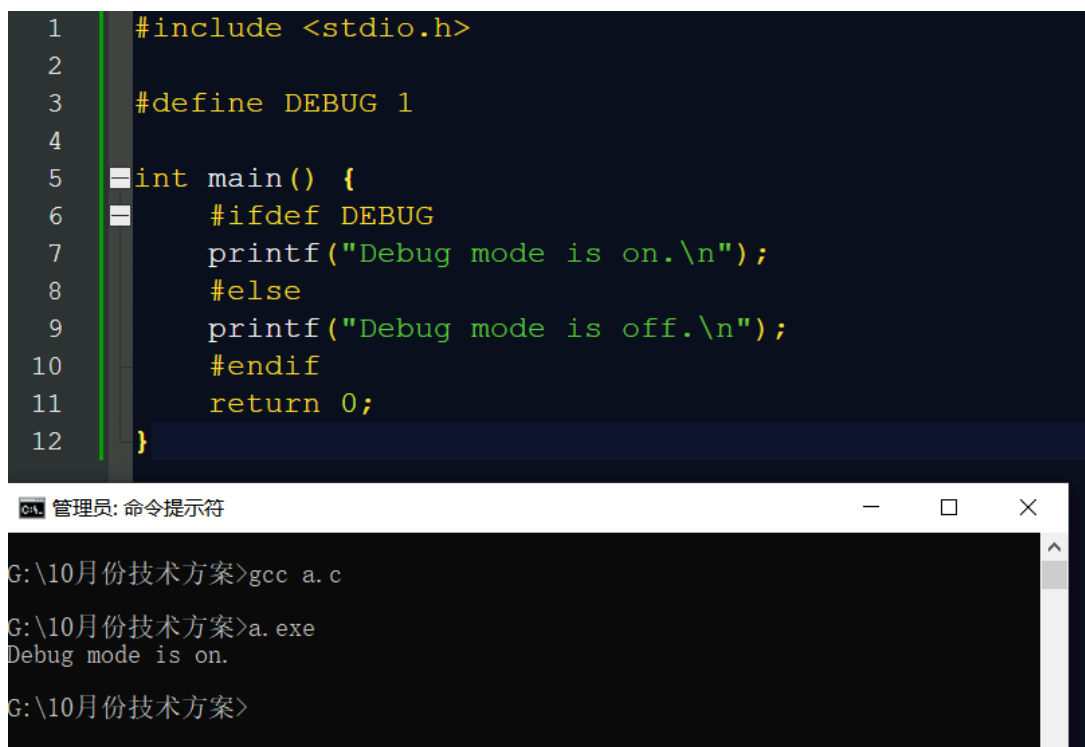
```
#include <stdio.h>

#define DEBUG 1

int main() {
    #ifdef DEBUG
        printf("Debug mode is on.\n");
    #else
        printf("Debug mode is off.\n");
    #endif
    return 0;
}
```

用法：`#ifdef` 和 `#ifndef` 指令用于条件编译。如果宏定义了，则编译 `#ifdef` 块中的代码；如果没有定义，则编译 `#else` 块中的代码。

输出：Debug mode is on.



```
1  #include <stdio.h>
2
3  #define DEBUG 1
4
5  int main() {
6      #ifdef DEBUG
7          printf("Debug mode is on.\n");
8      #else
9          printf("Debug mode is off.\n");
10     #endif
11     return 0;
12 }
```

管理员: 命令提示符

```
G:\10月份技术方案>gcc a.c
G:\10月份技术方案>a.exe
Debug mode is on.
G:\10月份技术方案>
```


1.11.5 #pragma 这里列出

在头文件中使用 `#pragma once` 可以避免在多个源文件中包含同一个头文件时出现重复定义的问题。

```
// myheader.h
#pragma once

#define LED_ON 1
#define LED_OFF 0
```

用法：`#pragma` 指令用于提供编译器特定的指令。`#pragma once` 通常用于头文件中，以防止头文件被重复包含。

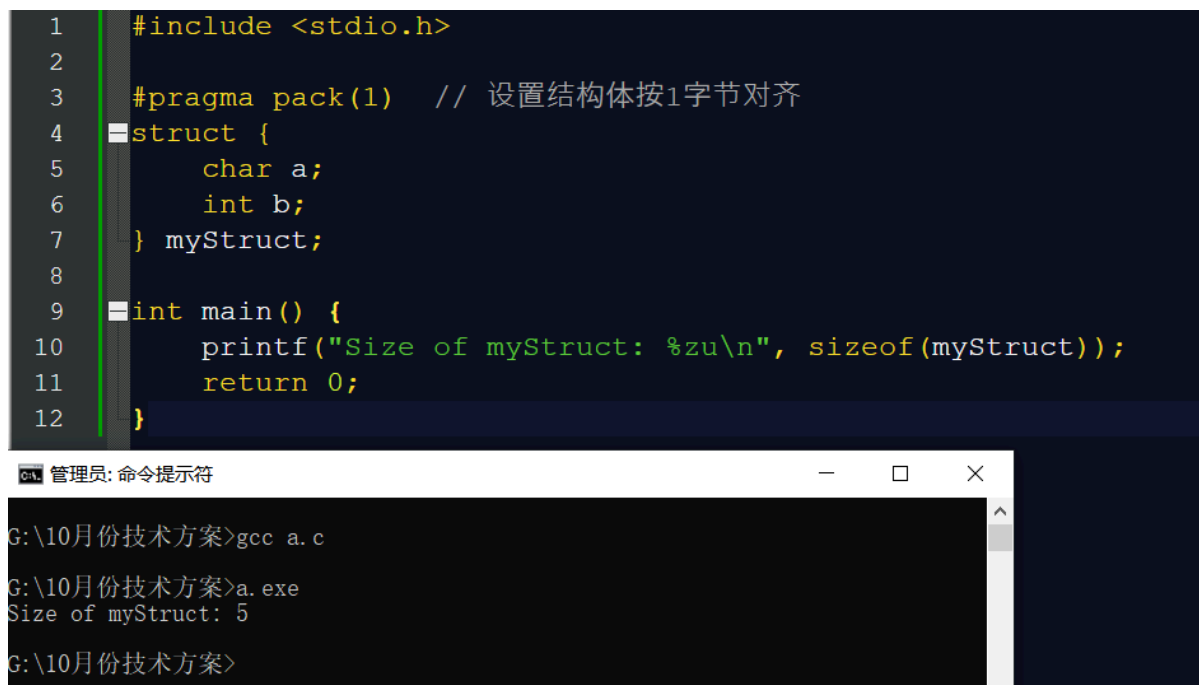
设置结构体按1字节对齐，这可以用于精确控制结构体的内存布局，以适应特定的硬件接口或协议要求。

```
#include <stdio.h>

#pragma pack(1) // 设置结构体按1字节对齐
struct {
    char a;
    int b;
} myStruct;

int main() {
    printf("Size of myStruct: %zu\n", sizeof(myStruct));
    return 0;
}
```

输出：Size of myStruct: 5



The screenshot shows a code editor with the following C code:

```
1  #include <stdio.h>
2
3  #pragma pack(1) // 设置结构体按1字节对齐
4  struct {
5      char a;
6      int b;
7  } myStruct;
8
9  int main() {
10     printf("Size of myStruct: %zu\n", sizeof(myStruct));
11     return 0;
12 }
```

Below the code editor is a Windows command prompt window titled "管理员: 命令提示符". It shows the following commands and output:

```
G:\10月份技术方案>gcc a.c
G:\10月份技术方案>a.exe
Size of myStruct: 5
G:\10月份技术方案>
```

```
1  #include <stdio.h>
2
3  // #pragma pack(1) // 设置结构体按1字节对齐
4  struct {
5      char a;
6      int b;
7  } myStruct;
8
9  int main() {
10     printf("Size of myStruct: %zu\n", sizeof(myStruct));
11     return 0;
12 }
```



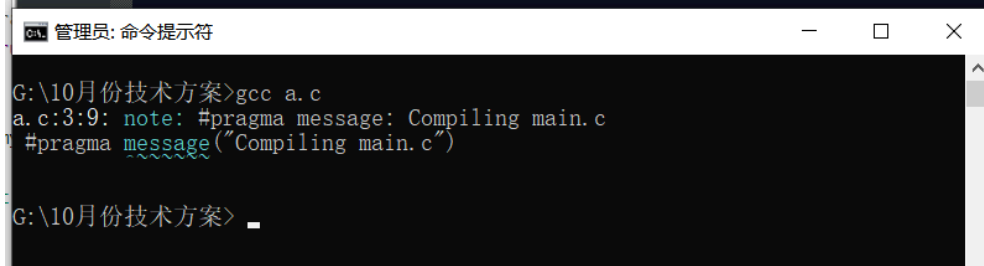
`#pragma message` 可以在编译时输出一条信息，这在STM32开发中可以用来显示编译时的状态信息或警告。

```
#include <stdio.h>

#pragma message("Compiling main.c")

int main() {
    printf("Hello, world!\n");
    return 0;
}
```

```
1  #include <stdio.h>
2
3  #pragma message("Compiling main.c")
4
5  int main() {
6      printf("Hello, World!\n");
7      return 0;
8  }
```



`#pragma warning`

用于控制编译器的警告信息。这可以用于关闭特定的编译器警告，特别是在使用某些库或硬件抽象层时。

示例代码：

```
#pragma warning(disable : 4996) // 禁用特定警告编号

#include <stdio.h>

int main() {
    printf("This is deprecated\n"); // 这行代码会产生警告
    return 0;
}
```

在这个例子中，`#pragma warning(disable : 4996)` 用于禁用特定的编译器警告，使得后续代码中的警告不再显示。

这些 `#pragma` 指令在STM32单片机编程中非常有用，可以帮助开发者更好地控制代码的编译和优化过程。

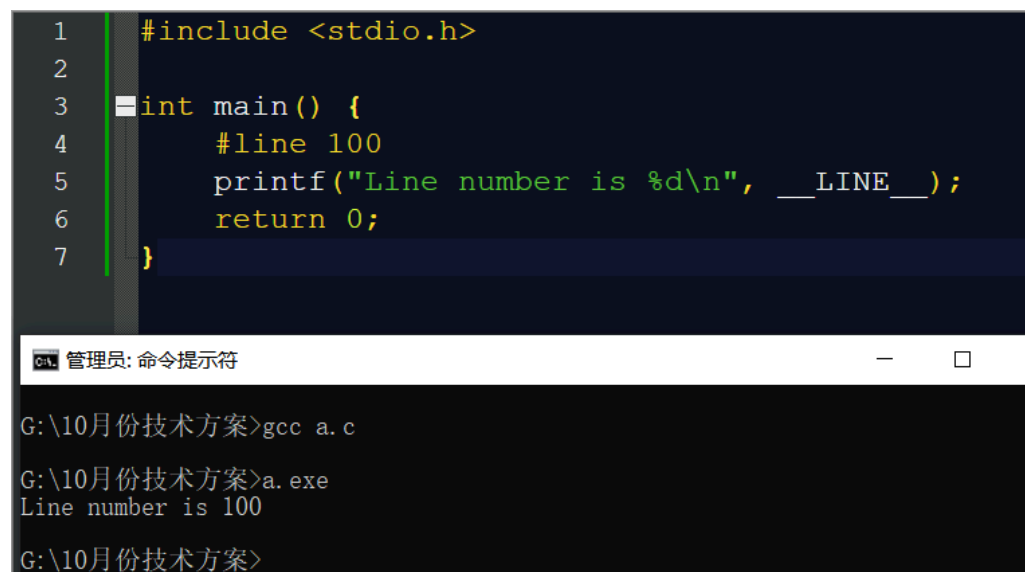
1.11.6 #line

```
#include <stdio.h>

int main() {
    #line 100
    printf("Line number is %d\n", __LINE__);
    return 0;
}
```

用法：`#line` 指令用于改变编译器报告的行号和文件名。这里我们将行号设置为 100。

输出：Line number is 100



The screenshot shows a code editor with the following C code:

```
1  #include <stdio.h>
2
3  int main() {
4      #line 100
5      printf("Line number is %d\n", __LINE__);
6      return 0;
7  }
```

Below the code editor is a Windows command prompt window titled "管理员: 命令提示符". It shows the following commands and output:

```
G:\10月份技术方案>gcc a.c
G:\10月份技术方案>a.exe
Line number is 100
G:\10月份技术方案>
```

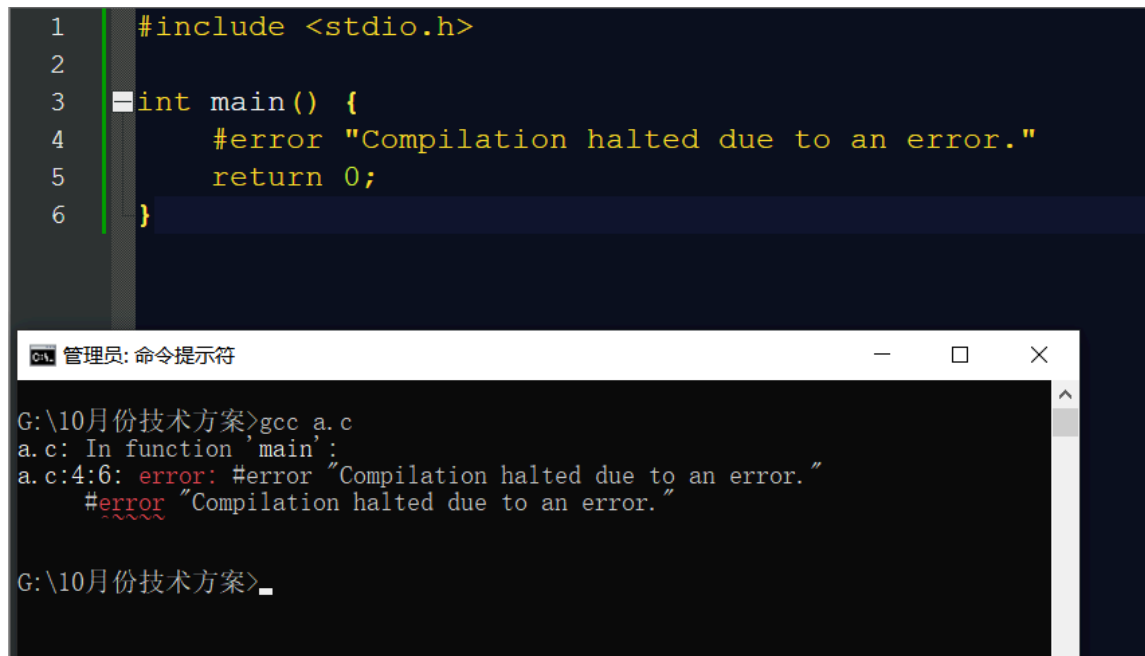
1.11.7 #error

```
#include <stdio.h>

int main() {
    #error "Compilation halted due to an error."
    return 0;
}
```

用法： `#error` 指令用于在编译时产生错误信息。

输出： 编译错误，错误信息为 `Compilation halted due to an error.`



The screenshot shows a code editor with the following C code:

```
1  #include <stdio.h>
2
3  int main() {
4      #error "Compilation halted due to an error."
5      return 0;
6  }
```

Below the code editor is a Windows command prompt window titled "管理员: 命令提示符". It shows the command `gcc a.c` being executed. The output is:

```
G:\10月份技术方案>gcc a.c
a.c: In function 'main':
a.c:4:6: error: #error "Compilation halted due to an error."
      #error "Compilation halted due to an error."
      ~~~~~
G:\10月份技术方案>_
```

1.11.8 #warning

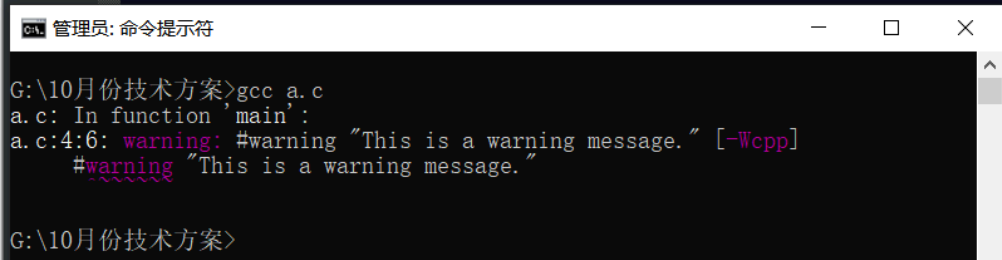
```
#include <stdio.h>

int main() {
    #warning "This is a warning message."
    printf("warning example.\n");
    return 0;
}
```

用法： `#warning` 指令用于在编译时产生警告信息。

输出： 编译警告，警告信息为 `This is a warning message.`，程序继续执行并输出 `warning example.`

```
1 #include <stdio.h>
2
3 int main() {
4     #warning "This is a warning message."
5     printf("Warning example.\n");
6     return 0;
7 }
```



这些示例展示了 C 语言预处理指令的基本用法和效果。预处理指令在编译过程中的第一阶段处理，它们不会影响程序的执行逻辑，但会影响编译过程和最终生成的代码。

2. 指针篇 - 更多是辅助梳理总结

2.1 指针是什么有什么作用

指针变量是存放地址的变量，通过地址来访问普通变量，访问数组，访问函数，访问结构体等

这种访问方式有以下优点和作用

2.1.1 动态内存管理

指针允许程序在运行时动态地分配和释放内存，这对于处理不确定大小的数据集或实现复杂的数据结构（如链表、树等）非常有用。

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *dynamicArray;
    int size = 10;

    // 动态分配数组
    dynamicArray = (int *)malloc(size * sizeof(int));
    if (dynamicArray == NULL) {
        fprintf(stderr, "Memory allocation failed.\n");
        return 1;
    }

    // 初始化数组
    for (int i = 0; i < size; i++) {
        dynamicArray[i] = i;
    }

    // 使用数组
    for (int i = 0; i < size; i++) {
        printf("%d ", dynamicArray[i]);
    }
}
```

```

    printf("\n");

    // 释放内存
    free(dynamicArray);

    return 0;
}

```

输出结果：

```

1 2 3 4 5 6 7 8 9 10

```

2.1.2 函数参数传递改值

通过指针，函数可以修改传入的变量的值，这在需要函数内部修改变量或返回多个值时非常有用。例如，函数可以通过指针参数直接修改数组的内容，或者返回一个结构体的地址。

```

#include <stdio.h>

void increment(int *num) {
    (*num) += 1;
}

int main() {
    int value = 5;
    printf("Before: %d\n", value);
    increment(&value);
    printf("After: %d\n", value);
    return 0;
}

```

输出结果：

```

Before: 5
After: 6

```

2.1.3 性能优化

使用指针可以避免数据的复制，特别是在处理大型数据结构时。这可以减少内存的使用和提高程序的运行效率。

案例是字符串拷贝，一个用指针，一个不用指针，计算拷贝时间。

```

#include <stdio.h>
#include <string.h>
#include <sys/time.h>

// 使用指针进行字符串复制
void copyStringWithPointer(char *dest, const char *src) {
    while ((*dest++ = *src++));
}

```

```

// 不使用指针进行字符串复制
void copyStringWithoutPointer(char dest[], const char src[]) {

    int i = 0;

    for ( i = 0; src[i] != '\0'; i++) {
        dest[i] = src[i];
    }
    dest[i] = '\0'; // 确保复制字符串结束符
}

// 测量函数执行时间的函数
double measureTime(void (*func)(char[], const char[]), char dest[], const char
src[]) {
    struct timeval start, end;
    double elapsedTime;

    gettimeofday(&start, NULL);
    func(dest, src); // 调用传入的函数
    gettimeofday(&end, NULL);

    elapsedTime = (end.tv_sec - start.tv_sec) * 1000.0; // sec to ms
    elapsedTime += (end.tv_usec - start.tv_usec) / 1000.0; // us to ms
    return elapsedTime;
}

int main() {
    char src[] = "Hello, world!";
    char dest[20];
    char destNoPointer[20];

    // 测量使用指针复制字符串的时间
    double timeTakenPointer = measureTime(copyStringWithPointer, dest, src);
    printf("Copied string with pointer: %s\n", dest);

    // 测量不使用指针复制字符串的时间
    double timeTakenNoPointer = measureTime(copyStringWithoutPointer,
destNoPointer, src);
    printf("Copied string without pointer: %s\n", destNoPointer);

    printf("Time taken with pointer: %f ms\n", timeTakenPointer);
    printf("Time taken without pointer: %f ms\n", timeTakenNoPointer);

    return 0;
}

```

输出结果:

```

Copied string with pointer: Hello, world!
Copied string without pointer: Hello, world!
Time taken with pointer: 0.007000 ms
Time taken without pointer: 0.008000 ms

```

```
#include <stdio.h>
#include <string.h>
#include <sys/time.h>

// 使用指针进行字符串复制
void copyStringWithPointer(char *dest, const char *src) {
    while ((*dest++ = *src++));
}

// 不使用指针进行字符串复制
void copyStringWithoutPointer(char dest[], const char src[]) {
    int i = 0;
    for (i = 0; src[i] != '\0'; i++) {
        dest[i] = src[i];
    }
    dest[i] = '\0'; // 确保复制字符串结束符
}

// 测量函数执行时间的函数
double measureTime(void (*func)(char[], const char[]), char dest[], const char src[]) {
    struct timeval start, end;
    double elapsedTime;

    gettimeofday(&start, NULL);
    func(dest, src); // 调用传入的函数
    gettimeofday(&end, NULL);

    elapsedTime = (end.tv_sec - start.tv_sec) * 1000.0; // sec to ms
    elapsedTime += (end.tv_usec - start.tv_usec) / 1000.0; // us to ms
    return elapsedTime;
}

int main() {
    char src[] = "Hello, World!";
    char dest[20];
    char destNoPointer[20];

    // 测量使用指针复制字符串的时间
    double timeTakenPointer = measureTime(copyStringWithPointer, dest, src);
    printf("Copied string with pointer: %s\n", dest);

    // 测量不使用指针复制字符串的时间
    double timeTakenNoPointer = measureTime(copyStringWithoutPointer, destNoPointer, src);
    printf("Copied string without pointer: %s\n", destNoPointer);

    printf("Time taken with pointer: %f ms\n", timeTakenPointer);
    printf("Time taken without pointer: %f ms\n", timeTakenNoPointer);

    return 0;
}
```

```
chen@H616:~/cTest$ ls
a.c  a.out
chen@H616:~/cTest$
chen@H616:~/cTest$
chen@H616:~/cTest$ gcc a.c
chen@H616:~/cTest$ ./a.out
Copied string with pointer: Hello, World!
Copied string without pointer: Hello, World!
Time taken with pointer: 0.007000 ms
Time taken without pointer: 0.008000 ms
chen@H616:~/cTest$
```

2.1.4 实现高级数据结构

许多高级数据结构，如链表、树、图等，都需要使用指针来连接数据元素。没有指针，这些数据结构的实现将变得非常困难或不可能。

简单的链表案例：

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

Node *createNode(int data) {
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == NULL) {
        fprintf(stderr, "Memory allocation failed.\n");
        return NULL;
    }
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

int main() {
    Node *head = createNode(1);
    head->next = createNode(2);
    head->next->next = createNode(3);

    Node *current = head;
    while (current != NULL) {
        printf("%d ", current->data);
        current = current->next;
    }
}
```



```

printf("\n");

// 释放链表内存
while (head != NULL) {
    current = head;
    head = head->next;
    free(current);
}

return 0;
}

```

输出

```
1 2 3
```

2.1.5 字符串操作

在C语言中，字符串通常通过字符数组或字符指针来处理。使用指针可以方便地遍历字符串、连接字符串以及操作字符串中的单个字符。

```

#include <stdio.h>

int main() {
    char str[] = "Chenlichen";
    char *ptr = str;

    // 打印字符串中的每个字符
    while (*ptr) {
        printf("%c", *ptr);
        ptr++;
    }
    printf("\n");

    return 0;
}

```

输出：

```
Chenlichen
```

2.1.6 函数返回多个值

函数可以通过返回指针来返回数组或动态分配的内存块，从而允许函数返回多个值或大量数据。

```

#include <stdio.h>

int *maxAndMin(int a, int b) {
    static int result[2];
    result[0] = (a > b) ? a : b;
}

```

```

    result[1] = (a > b) ? b : a;
    return result;
}

int main() {
    int maxVal, minVal;
    int *result = maxAndMin(10, 20);
    maxVal = result[0];
    minVal = result[1];
    printf("Max: %d, Min: %d\n", maxVal, minVal);
    return 0;
}

```

输出:

```
Max: 20, Min: 10
```

2.1.7 灵活性和控制

指针提供了对内存的直接控制，使得程序员可以精确地管理内存布局和访问方式，这在嵌入式编程和系统级编程中尤为重要。

```

#include <stdio.h>

int main() {
    int array[] = {10, 20, 30, 40, 50};
    int *ptr = array; // 指针指向数组的开始
    for (int i = 0; i < sizeof(array)/sizeof(array[0]); ++i) {
        *(ptr + i) = *(ptr + i) * 2; // 通过指针访问和修改数组元素
    }

    for (int i = 0; i < sizeof(array)/sizeof(array[0]); ++i) {
        printf("%d ", array[i]);
    }
    printf("\n");

    return 0;
}

```

输出:

```
20 40 60 80 100
```

2.1.8 与C语言库的兼容性

许多C语言的标准库函数，如 `malloc`、`free`、`realloc` 等，都依赖于指针来管理动态内存。熟悉指针对于使用这些库函数至关重要。

```

#include <stdio.h>
#include <stdlib.h>

int main() {

```

```

int *dynamicArray = (int *)malloc(5 * sizeof(int));
if (dynamicArray == NULL) {
    fprintf(stderr, "Memory allocation failed.\n");
    return 1;
}

// 初始化动态分配的数组
for (int i = 0; i < 5; ++i) {
    dynamicArray[i] = i;
}

// 释放动态分配的内存
free(dynamicArray);
printf("Memory freed.\n");

return 0;
}

```

输出：

```
Memory freed.
```

2.1.9 优化数组操作

指针可以用于简化数组的遍历和操作，使得对数组的处理更加直观和灵活。

```

#include <stdio.h>

void printArray(int *array, int size) {    //如果没有指针, void printArray(int
array[], int size),看着怪怪的
    for (int i = 0; i < size; ++i) {
        printf("%d ", array[i]);
    }
    printf("\n");
}

int main() {
    int myArray[] = {1, 2, 3, 4, 5};
    printArray(myArray, sizeof(myArray)/sizeof(myArray[0]));
    return 0;
}

```

2.1.10 接口硬件

在与硬件接口编程时，指针经常用于访问和控制硬件寄存器，实现硬件的直接操作。

```

#include <stdio.h>

// 模拟硬件寄存器的地址
#define HARDWARE_REGISTER (*(volatile unsigned int *)0x40021000)

void setHardwareRegister(unsigned int value) {
    HARDWARE_REGISTER = value; // 设置硬件寄存器的值
}

```

```

unsigned int getHardwareRegister() {
    return HARDWARE_REGISTER; // 获取硬件寄存器的值
}

int main() {
    // 向硬件寄存器写入值
    setHardwareRegister(123);
    // 读取硬件寄存器的值
    unsigned int value = getHardwareRegister();
    printf("Hardware register value: %u\n", value);
    return 0;
}

```

2.2 指针和变量

- **指针变量**：存储其他变量地址的变量。
- **指针的类型**：必须与它指向的变量类型匹配。
- **解引用**：通过指针访问其指向的变量的值。

2.2.1 指针和变量的关系

- **地址操作符 (&)**：用于获取变量的内存地址，并可以将其赋值给一个指针。
- **解引用操作符 (*)**：用于访问指针指向的变量的值。

```

#include <stdio.h>

int main() {
    int var = 10; // 定义一个整型变量var
    int *ptr;     // 定义一个整型指针ptr

    ptr = &var; // 将ptr指向var的地址

    printf("Value of var: %d\n", var); // 直接访问变量
    printf("Address of var: %p\n", &var); // 获取变量的地址
    printf("Value at address ptr points to: %d\n", *ptr); // 通过指针访问变量的值

    *ptr = 20; // 通过指针修改变量的值
    printf("New value of var: %d\n", var); // 变量的值被修改

    return 0;
}

```

在这个程序中，我们定义了一个变量 `var` 和一个指针 `ptr`。我们使用 `&` 操作符获取 `var` 的地址，并将其赋值给 `ptr`。然后，我们使用 `*` 操作符通过 `ptr` 访问和修改 `var` 的值。

输出：

```

value of var: 10
Address of var: 0x7ffeedfffff8
value at address ptr points to: 10
New value of var: 20

```

2.3 指针数组

2.3.1 它是一个数组，数组的每一项都是一个指针。

```
#include <stdio.h>

int main() {
    int a = 10;
    int b = 20;
    int c = 30;
    int *ptrArray[] = {&a, &b, &c}; // 指针数组，存储指向整数的指针

    for (int i = 0; i < 3; i++) {
        printf("value at ptrArray[%d]: %d\n", i, *ptrArray[i]);
    }

    return 0;
}
```

输出：

```
value at ptrArray[0]: 10
value at ptrArray[1]: 20
value at ptrArray[2]: 30
```

实际应用当中比较多的是，数组中每一项都是一个函数的地址，又叫做函数指针数组。

比如单片机上电后要初始化，把函数的地址放在数组中管理，然后用遍历数组的方式获取每个函数的地址，通过地址来调用函数

```
#include <stdio.h>

// 函数声明
void greet(const char *message) {
    printf("%s\n", message);
}

void farewell(const char *message) {
    printf("%s\n", message);
}

int main() {
    // 函数指针数组
    void (*funcArray[])(const char *) = {greet, farewell};

    // 调用函数指针数组中的函数
    funcArray[0]("Hello, world!");
    funcArray[1]("Goodbye, world!");

    return 0;
}
```

输出

```
Hello, world!  
Goodbye, world!
```

2.4 数组指针

2.4.1 它是一个指针，指向一个固定大小的数组

它是一个指针，指向一个数组的首元素。

数组指针的概念与指针数组不同，

- 指针数组是包含多个指针的数组
- 数组指针是一个单独的指针，它指向一个数组。

数组指针常用于函数参数中，特别是当你需要传递一个数组给函数，并且希望函数能够处理整个数组时。使用数组指针可以避免复制整个数组，从而节省内存和提高效率。

声明一个数组指针时，需要指定数组的元素类型和大小。例如，指向一个整数数组的指针可以这样声明：

```
int (*arrayPtr)[10]; // 指向一个包含10个整数的数组的指针
```

这里，`arrayPtr` 是一个指针，它指向一个包含10个整数的数组。

以下是使用数组指针的示例代码：

```
#include <stdio.h>  
  
// 使用数组指针作为形式参数的函数  
void printArray(int (*arrPtr)[5], int size) {  
    for (int i = 0; i < size; i++) {  
        printf("%d ", (*arrPtr)[i]); // 通过数组指针访问数组元素  
    }  
    printf("\n");  
}  
  
int main() {  
    int myArray[5] = {1, 2, 3, 4, 5};  
    int (*arrayPtr)[5] = &myArray; // arrayPtr 指向 myArray  
  
    printf("Using array pointer:\n");  
    printArray(arrayPtr, 5); // 传递数组指针给函数  
  
    return 0;  
}
```

输出结果：

```
Using array pointer:  
1 2 3 4 5
```

2.4.2 数组指针和二维数组

示例：数组指针作为形参！实际参数是二维数组的数组名，用的比较多。

```
#include <stdio.h>

#define ROWS 3
#define COLS 5

// 打印二维数组的函数
void print2DArray(int (*arrPtr)[COLS], int rows, int cols) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%d ", (*arrPtr)[i][j]);
        }
        printf("\n");
    }
}

int main() {
    int myArray[ROWS][COLS] = {
        {1, 2, 3, 4, 5},
        {6, 7, 8, 9, 10},
        {11, 12, 13, 14, 15}
    };

    print2DArray(myArray, ROWS, COLS);

    return 0;
}
```

输出

```
1 2 3 4 5
6 7 8 9 10
11 12 13 14 15
```

2.5 指针函数-返回指针的函数

实际上，返回指针的函数通常被称为“返回指针的函数”或者简称为“返回指针”的函数，有的文章不认“指针函数”的说法，我们知道就行。

在C语言中，函数可以返回各种类型的值，包括指针类型。这种函数通常用于动态内存分配后返回内存块的地址，或者返回静态或全局变量的地址。

以下是一些关于返回指针的函数的示例：

2.5.1 返回局部变量的指针（不推荐）

```
#include <stdio.h>

int *func() {
    int localVar = 10;
    return &localVar; // 警告：返回局部变量的地址
}

int main() {
    int *ptr = func();
    printf("%d\n", *ptr); // 未定义行为，因为局部变量localVar的存储空间在func返回后被释放
    return 0;
}
```

注意：这个示例是不安全的，因为函数返回了局部变量的地址，该地址在函数返回后不再有效，导致未定义行为。

2.5.2 返回动态分配的内存地址

```
#include <stdio.h>
#include <stdlib.h>

int *createArray(int size) {
    int *array = malloc(size * sizeof(int));
    if (array == NULL) {
        fprintf(stderr, "Memory allocation failed.\n");
        return NULL;
    }
    // 初始化数组
    for (int i = 0; i < size; i++) {
        array[i] = i;
    }
    return array; // 返回动态分配的内存地址
}

int main() {
    int *myArray = createArray(5);
    if (myArray != NULL) {
        for (int i = 0; i < 5; i++) {
            printf("%d ", myArray[i]);
        }
        printf("\n");
        free(myArray); // 释放内存
    }
    return 0;
}
```

输出结果：

```
0 1 2 3 4
```

在这个示例中，`createArray` 函数动态分配了一个数组，并返回了这个数组的指针。`main` 函数接收这个指针，使用它访问数组元素，并最终释放内存。

2.5.3 返回静态变量的指针

```
#include <stdio.h>

static int staticVar = 10;

int *getStaticVarPtr() {
    return &staticVar; // 返回静态变量的地址
}

int main() {
    int *ptr = getStaticVarPtr();
    printf("%d\n", *ptr); // 安全地访问静态变量
    return 0;
}
```

输出结果：

```
10
```

在这个示例中，`getStaticVarPtr` 函数返回了一个指向静态变量 `staticVar` 的指针。因为静态变量的生命周期贯穿整个程序运行期，所以这个指针在 `getStaticVarPtr` 返回后仍然有效。

这些示例展示了返回指针的函数的不同用法和注意事项。在使用这类函数时，特别要注意内存管理和指针的有效性，以避免未定义行为和内存泄漏。

2.6 函数指针-回调函数的应用

函数指针是指向函数的指针，它允许你将函数作为参数传递给另一个函数，或者从函数中返回函数。函数指针在C语言中被广泛用于回调函数、事件处理、多态和接口实现等场景。

声明函数指针时，需要指定指向的函数的返回类型、参数类型以及函数的名称。例如，声明一个指向返回 `int` 并接受两个 `int` 参数的函数的指针如下：

```
int (*functionPointer)(int, int);
```

这里，`functionPointer` 是一个函数指针，它指向的函数返回 `int` 类型，并且接受两个 `int` 类型的参数。

2.6.1 使用函数指针

以下是使用函数指针的示例代码：

```
#include <stdio.h>

// 定义两个函数，它们具有相同的签名
int add(int a, int b) {
    return a + b;
}

int subtract(int a, int b) {
    return a - b;
}
```

```

int main() {
    // 声明并初始化函数指针
    int (*operation)(int, int) = add; // 初始指向 add 函数

    // 使用函数指针调用函数
    printf("10 + 5 = %d\n", operation(10, 5)); // 输出结果

    // 更改函数指针指向的函数
    operation = subtract;
    printf("10 - 5 = %d\n", operation(10, 5)); // 输出结果

    return 0;
}

```

输出结果：

```

10 + 5 = 15
10 - 5 = 5

```

在这个程序中，`operation` 是一个函数指针，它最初指向 `add` 函数。我们通过函数指针调用 `add` 函数，并打印结果。然后，我们将 `operation` 指向 `subtract` 函数，并再次调用。

2.6.2 函数指针作为参数-回调函数

函数指针可以作为参数传递给其他函数，这允许实现回调机制：

```

#include <stdio.h>

// 定义一个接受函数指针（指向返回int并接受两个int参数的函数）作为参数的函数
void performOperation(int (*func)(int, int), int a, int b) {
    printf("Result: %d\n", func(a, b));
}

int add(int a, int b) {
    return a + b;
}

int main() {
    // 将 add 函数作为参数传递，其类型与 performOperation 期望的函数指针类型匹配
    performOperation(add, 10, 5);
    return 0;
}

```

输出结果：

```

Result: 15

```

在这个程序中，`performOperation` 函数接受函数指针 `func` 作为参数，并使用它来计算和打印结果。`main` 函数中将 `add` 函数作为参数传递给 `performOperation`。

函数指针是C语言中实现多态和回调机制的重要工具，它们提供了代码的灵活性和模块化。

2.7 指针和二维数组

2.7.1 指针与二维数组的关系

1. 二维数组的内存布局：

二维数组在内存中实际上是连续存储的。例如，`int array[3][4]`；声明了一个3行4列的二维数组，它在内存中就像是一个包含12个整数的一维数组。

2. 数组名作为指针：

在大多数情况下，数组名会被解释为指向数组第一个元素的指针。对于二维数组，数组名可以被视为指向其第一行（即一个一维数组）的指针。例如，`array` 可以被视为指向 `int[4]` 的指针。

3. 访问二维数组的元素：

使用数组名和两个索引（如 `array[i][j]`）来访问二维数组的元素时，C语言会根据数组的维度来计算正确的内存偏移量。

4. 指向数组的指针：

我们可以声明一个指向数组的指针来访问二维数组的元素。例如，`int (*ptr)[4]`；声明了一个指向包含4个整数的数组的指针。通过 `ptr`，我们可以访问二维数组的一行。

5. 动态分配二维数组：

使用指针和动态内存分配（如 `malloc`）来创建二维数组可以提供更大的灵活性。例如，我们可以先分配一个指针数组，然后为每个指针分配一个一维数组的空间。

2.7.2 使用数组名访问二维数组

```
#include <stdio.h>

int main() {
    int array[3][4] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };

    // 使用数组名和两个索引访问元素
    printf("%d\n", array[1][2]); // 输出7

    return 0;
}
```

2.7.3 使用指向数组的指针访问二维数组

```
#include <stdio.h>

int main() {
    int array[3][4] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };

    // 声明一个指向数组的指针
    int (*ptr)[4] = array; //第2.4.2章节类似

    // 使用指针访问元素
```

```

printf("%d\n", (*ptr)[1][2]); // 输出7，等同于array[1][2]

// 或者使用指针算术来访问元素
printf("%d\n", ptr[1][2]); // 直接输出7

return 0;
}

```

72.7.4 动态分配二维数组

这个案例，多看，多思考，也许对指针的理解会更进一步，或者直接先看下面一节-二级指针

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    int rows = 3, cols = 4;
    int **array = (int **)malloc(rows * sizeof(int *));

    for (int i = 0; i < rows; i++) {
        array[i] = (int *)malloc(cols * sizeof(int));
    }

    // 初始化数组元素
    int value = 1;
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            array[i][j] = value++;
        }
    }

    // 打印数组元素
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%d ", array[i][j]);
        }
        printf("\n");
    }

    // 释放内存
    for (int i = 0; i < rows; i++) {
        free(array[i]);
    }
    free(array);

    return 0;
}

```

在这个示例中，我们使用 malloc 动态地分配了一个二维数组，并在使用完毕后释放了分配的内存。

2.8 二级指针

二级指针是指向指针的指针，在C语言中，它是一种特殊的指针类型，用于指向另一个指针变量的地址。二级指针通常用于函数参数传递、动态内存管理、数据结构（如链表、树等）的实现等场景。

2.8.1 定义二级指针

在C语言中，定义一个二级指针的语法如下：

```
int **ptr;
```

这里，`ptr` 是一个二级指针，它指向一个 `int*` 类型的指针。

2.8.2 初始化二级指针

初始化二级指针时，需要先为它分配一个指针变量的地址：

```
int var = 10; // 定义一个整型变量
int *ptr1 = &var; // ptr1 是一个一级指针，指向 var 的地址
int **ptr2 = &ptr1; // ptr2 是一个二级指针，指向 ptr1 的地址
```

2.8.3 使用二级指针

使用二级指针时，可以通过解引用来访问它指向的一级指针指向的值：

```
#include <stdio.h>

int main() {
    int var = 10; // 定义一个整型变量
    int *ptr1 = &var; // ptr1 是一个一级指针，指向 var 的地址
    int **ptr2 = &ptr1; // ptr2 是一个二级指针，指向 ptr1 的地址

    // 通过二级指针访问变量var的值
    printf("The value of var is: %d\n", **ptr2);

    return 0;
}
```

2.8.4 动态内存分配

二级指针经常用于动态内存分配，例如：

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int **array = (int **)malloc(10 * sizeof(int*)); // 分配10个int指针的空间
    if (array == NULL) {
        fprintf(stderr, "Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 10; i++) {
        array[i] = malloc(10 * sizeof(int)); // 为每个指针分配一个int数组的空间
        if (array[i] == NULL) {
```

```

        fprintf(stderr, "Memory allocation failed for array[%d]\n", i);
        // 释放之前已分配的内存
        for (int j = 0; j < i; j++) {
            free(array[j]);
        }
        free(array);
        return 1;
    }
    // 初始化数组
    for (int j = 0; j < 10; j++) {
        array[i][j] = i * 10 + j;
    }
}

// 打印数组内容
for (int i = 0; i < 10; i++) {
    for (int j = 0; j < 10; j++) {
        printf("%d ", array[i][j]);
    }
    printf("\n");
}

// 释放内存
for (int i = 0; i < 10; i++) {
    free(array[i]);
}
free(array);

return 0;
}

```

在这个例子中，`array` 是一个二级指针，它指向一个包含10个一级指针的数组，每个一级指针又指向一个包含10个整数的数组。

注意事项

- 二级指针在使用前必须被正确初始化，否则可能会导致未定义行为。
- 在使用动态内存分配时，记得释放分配的内存，避免内存泄漏。
- 二级指针可以用于实现复杂的数据结构，但也需要更多的注意来管理内存和指针。

二级指针是C语言中一个强大的特性，但也需要谨慎使用，以避免常见的指针错误。

2.9 指针和结构体

指针和结构体在C语言中是两个非常重要的概念，它们经常一起使用来创建复杂的数据结构。

2.9.1 使用指针和结构体管理学生信息

这个示例中，我们将创建一个学生信息的结构体，然后使用指针来操作这些信息。

有的同学会对用 `.` 还是用 `->` 造成混乱，如果是指针的方式访问结构体的元素，用 `ptr->age`

```

#include <stdio.h>
#include <stdlib.h>

// 定义学生信息的结构体

```

```

typedef struct Student {
    char name[50];
    int age;
    float score;
} Student;

int main() {
    // 创建一个学生信息的实例
    Student stu;
    // 使用指针指向这个学生信息
    Student *ptr = &stu;

    // 通过指针给结构体成员赋值
    printf("Enter student's name: ");
    fgets(ptr->name, 50, stdin);
    ptr->name[strcspn(ptr->name, "\n")] = 0; // 去除换行符

    printf("Enter student's age: ");
    scanf("%d", &ptr->age);
    getchar(); // 消耗换行符

    printf("Enter student's score: ");
    scanf("%f", &ptr->score);
    getchar(); // 消耗换行符

    // 通过指针访问结构体成员并打印
    printf("\nStudent Information:\n");
    printf("Name: %s\n", ptr->name);
    printf("Age: %d\n", ptr->age);
    printf("Score: %.2f\n", ptr->score);

    return 0;
}

```

2.10 野指针 - 面试常见

在C语言中，野指针（Wild Pointer）指的是一个未初始化或已经被释放的指针。

野指针可能指向任何地方，包括无效的内存地址，因此使用野指针可能会导致程序崩溃或者数据损坏。

2.10.1 野指针的成因

1. **未初始化的指针**：声明指针后未对其进行初始化，其值可能是任意的。
2. **释放后的指针**：指针指向的内存已经被释放，但指针未被置为NULL。
3. **函数返回局部变量的地址**：函数返回局部变量的地址，一旦函数执行结束，局部变量的内存被释放，指针成为野指针。
4. **数组越界**：指针访问数组之外的内存，可能会成为野指针。

2.10.2 野指针的危害

- **程序崩溃**：野指针可能会导致程序试图访问无效的内存地址，从而触发段错误（Segmentation Fault）。
- **数据损坏**：野指针可能会覆盖重要的数据，导致数据损坏。
- **安全漏洞**：野指针可能被利用作为安全攻击的入口，比如缓冲区溢出攻击。

2.10.3 避免野指针的措施

1. **初始化指针**：声明指针后立即初始化，例如 `int *ptr = NULL;`。
2. **释放内存后置NULL**：释放指针指向的内存后，立即将指针置为NULL，例如 `free(ptr); ptr = NULL;`。
3. **使用动态内存分配时检查返回值**：在使用 `malloc`、`calloc` 或 `realloc` 等函数分配内存时，检查返回值是否为NULL。
4. **避免返回局部变量的地址**：不要返回局部变量的地址，如果需要返回数据，考虑使用动态内存分配或者传递指针参数。
5. **数组访问时检查边界**：在访问数组元素时，总是检查索引是否在有效范围内。

2.10.4 演示野指针

```
#include <stdio.h>

int main() {
    int *ptr; // 未初始化的指针

    // 尝试解引用未初始化的指针
    printf("Value pointed to by ptr: %d\n", *ptr);

    return 0;
}
```

上述代码中，`ptr` 是一个未初始化的指针，尝试解引用它将导致未定义行为，通常是程序崩溃。

2.10.5 安全的代码示例

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = NULL; // 初始化指针为NULL

    // 动态分配内存
    ptr = (int *)malloc(sizeof(int));
    if (ptr == NULL) {
        fprintf(stderr, "Memory allocation failed\n");
        return 1;
    }

    *ptr = 10; // 安全地使用指针

    printf("Value pointed to by ptr: %d\n", *ptr);

    // 释放内存并置NULL!!!!!!!!!!!!!! 面试也有可能提起
    free(ptr);
    ptr = NULL;

    return 0;
}
```


在这个安全的示例中，我们初始化了指针，检查了动态内存分配的返回值，并在释放内存后将指针置为 NULL，这些都是避免野指针的好习惯。

2.11 内存泄露 - 面试常见

内存泄漏（Memory Leak）是指在程序中动态分配的内存没有得到正确释放，导致随着时间的推移，分配的内存无法再被访问或使用，最终可能耗尽系统内存，影响程序的稳定性和性能。

2.11.1 内存泄漏的成因

1. **未释放已分配的内存**：使用 `malloc`、`calloc` 或 `realloc` 等函数分配了内存，但在不再需要时没有使用 `free` 函数释放。
2. **丢失对已分配内存的引用**：分配了内存但丢失了对这块内存的引用，导致无法释放。
3. **异常路径导致内存未释放**：在代码中存在异常路径（如错误处理、提前返回等），在这些路径中没有释放内存。
4. **循环引用**：在复杂的数据结构中，如链表或树，节点之间的循环引用可能导致内存无法被垃圾回收器回收。

2.11.2 内存泄漏的危害

- **程序性能下降**：随着内存泄漏的增加，程序可用内存减少，可能导致程序运行缓慢。
- **系统稳定性影响**：严重的内存泄漏可能导致系统内存耗尽，引发系统崩溃。
- **资源浪费**：内存泄漏导致宝贵的内存资源被无效占用，无法用于其他任务。

2.11.3 避免内存泄漏的措施

1. **及时释放内存**：确保每次 `malloc`、`calloc` 或 `realloc` 后都有相应的 `free` 调用来释放内存。
2. **使用智能指针**：在支持智能指针的语言中（如C++），使用智能指针自动管理内存。
3. **检查所有异常路径**：确保在所有可能的代码路径中，已分配的内存都能被正确释放。
4. **使用内存泄漏检测工具**：使用如Valgrind、LeakSanitizer等工具检测内存泄漏。
5. **避免循环引用**：在复杂的数据结构中，确保及时断开循环引用，或使用垃圾回收机制。

2.11.4 演示内存泄漏

```
#include <stdio.h>
#include <stdlib.h>

void someFunction() {
    int *ptr = (int *)malloc(sizeof(int)); // 分配内存
    *ptr = 10; // 使用内存
    // 没有释放内存
}

int main() {
    for (int i = 0; i < 1000; i++) {
        someFunction(); // 多次调用函数，每次都会泄漏内存
    }
    return 0;
}
```

在这个示例中，`someFunction` 函数分配了内存但没有释放，导致每次调用都会发生内存泄漏。

2.11.5 安全的代码示例

```
#include <stdio.h>
#include <stdlib.h>

void safeFunction() {
    int *ptr = (int *)malloc(sizeof(int)); // 分配内存
    if (ptr == NULL) {
        fprintf(stderr, "Memory allocation failed\n");
        return;
    }
    *ptr = 10; // 使用内存
    free(ptr); // 释放内存
}

int main() {
    for (int i = 0; i < 1000; i++) {
        safeFunction(); // 每次调用都会正确释放内存
    }
    return 0;
}
```

在这个安全的示例中，`safeFunction` 函数在分配内存后，无论是否成功，都会确保在函数结束前释放内存，从而避免了内存泄漏。

通过这些措施和示例，可以更好地理解和避免内存泄漏，确保程序的稳定性和性能。

3. 高频出现的编程题篇 - 其实可以直接背下记住

3.1 冒泡排序

(笔试面试阶段，我们直接记住两个for循环，i和j的值如何确定)

冒泡排序的基本思想是：

每次比较两个相邻的元素，如果他们的顺序错误就把他们交换过来。遍历数列的工作是重复进行直到没有再需要交换，也就是说该数列已经排序完成。

3.1.1 算法步骤

1. 比较相邻的元素。如果第一个比第二个大，就交换他们两个。
2. 对每一对相邻元素做同样的工作，从开始第一对到结尾的最后一对。这步做完后，最后的元素会是最大的数。
3. 针对所有的元素重复以上的步骤，除了最后一个。
4. 重复步骤1~3，直到排序完成。

3.1.2 代码实现

```
#include <stdio.h>

void bubbleSort(int arr[], int n) {
    int i, j;
    for (i = 0; i < n-1; i++) {
```

```

        // 最后的元素们已经在位
        for (j = 0; j < n-i-1; j++) {
            if (arr[j] > arr[j+1]) {
                // 相邻元素大小关系错误，交换之
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}

void printArray(int arr[], int size) {
    int i;
    for (i = 0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr)/sizeof(arr[0]);
    bubbleSort(arr, n);
    printf("Sorted array: \n");
    printArray(arr, n);
    return 0;
}

```

3.1.3 冒泡排序的特点

- **时间复杂度：**（这个记忆下，防止被问）平均和最坏情况下的时间复杂度都是 $O(n^2)$ ，其中 n 是数列的长度。最好情况下（数列已经是有序的）时间复杂度是 $O(n)$ 。
- **空间复杂度：**（这个记忆下，防止被问） $O(1)$ ，因为它只需要一个额外的存储空间来交换元素。
- **稳定性：**冒泡排序是稳定的排序算法，因为它不会改变相同元素之间的顺序。
- **适用性：**由于其低效率，冒泡排序不适合数据量大的排序任务。它更适用于数据量小或者基本有序的数据。

3.2 选择排序

它的工作原理是首先在未排序序列中找到最小（或最大）元素，将其存放到排序序列的起始位置，然后，再从剩余未排序元素中继续寻找最小（或最大）元素，然后放到已排序序列的末尾。以此类推，直到所有元素均排序完毕。

3.2.1 算法步骤

1. 从待排序的数据元素中选出最小（或最大）的一个元素，存放在序列的起始位置。
2. 然后再从剩余未排序元素中寻找最小（或最大）元素，然后放到已排序序列的末尾。
3. 以此类推，直到所有元素均排序完毕。

3.2.2 代码实现

```
#include <stdio.h>

void selectionSort(int arr[], int n) {
    int i, j, min_idx, temp;

    // 移动未排序数组的边界
    for (i = 0; i < n-1; i++) {
        // 找到未排序部分的最小元素
        min_idx = i;
        for (j = i+1; j < n; j++) {
            if (arr[j] < arr[min_idx]) {
                min_idx = j;
            }
        }

        // 将找到的最小元素与未排序部分的第一个元素交换
        temp = arr[min_idx];
        arr[min_idx] = arr[i];
        arr[i] = temp;
    }
}

void printArray(int arr[], int size) {
    int i;
    for (i = 0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr)/sizeof(arr[0]);
    selectionSort(arr, n);
    printf("Sorted array: \n");
    printArray(arr, n);
    return 0;
}
```

3.2.3 选择排序的特点

- **时间复杂度**：无论数组是否已经排序，选择排序的时间复杂度都是 $O(n^2)$ ，其中 n 是数组的长度。
- **空间复杂度**： $O(1)$ ，因为它只需要一个额外的存储空间来交换元素。
- **稳定性**：选择排序是不稳定的排序算法，因为它可能会改变相同元素之间的顺序。
- **适用性**：由于其低效率，选择排序不适合数据量大的排序任务。它适合于数据量小或者内存空间受限的情况。

3.3 字符串拷贝

3.3.1 代码实现

```
#include <stdio.h>

void stringCopy(char *dest, const char *src) {
    while (*src) {
        *dest = *src;
        src++;
        dest++;
    }
    *dest = '\0'; // 在目标字符串的末尾添加空字符
}

int main() {
    char src[] = "Hello, world!";
    char dest[50] = {0}; // 确保有足够的空间来存储源字符串

    stringCopy(dest, src);

    printf("Source: %s\n", src);
    printf("Destination: %s\n", dest);

    return 0;
}
```

3.3.2 注意事项

1. **缓冲区溢出**：在手动实现字符串拷贝时，必须确保目标数组有足够的空间来存储源字符串，否则可能会导致缓冲区溢出，这是一个常见的安全问题。
2. **空终止符**：在字符串的末尾必须添加空字符（`\0`），这是C语言字符串的标准结束标志。

3.4 字符串分割

字符串分割是指将一个字符串分割成多个子字符串的过程。在C语言中，没有直接提供字符串分割的函数，但可以通过编写自定义函数来实现。

3.4.1 代码实现

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/**
 * 分割字符串的函数
 * @param str 需要分割的字符串
 * @param delimiter 分隔符
 * @return 分割后的字符串数组，需要调用者负责释放内存
 */
char** split(const char* str, const char* delimiter, int* count) {
    int delimiterLen = strlen(delimiter);
    int strLen = strlen(str);
    int numTokens = 0;
```

```

int tokensSize = 10;
char** tokens = malloc(tokensSize * sizeof(char*));

char* tempStr = strdup(str); // 创建str的副本，因为strtok会修改原始字符串
char* token = strtok(tempStr, delimiter);

while (token) {
    tokens[numTokens++] = strdup(token);
    if (numTokens >= tokensSize) {
        tokensSize *= 2;
        tokens = realloc(tokens, tokensSize * sizeof(char*));
    }
    token = strtok(NULL, delimiter);
}

tokens[numTokens] = NULL; // 确保最后一个元素是NULL，表示字符串数组的结束

*count = numTokens;
free(tempStr); // 释放临时字符串的内存
return tokens;
}

int main() {
    const char* str = "one,two,three,four,five";
    const char* delimiter = ",";
    int count;

    char** tokens = split(str, delimiter, &count);

    printf("Split result:\n");
    for (int i = 0; i < count; i++) {
        printf("%s\n", tokens[i]);
        free(tokens[i]); // 释放每个分割出来的字符串的内存
    }
    free(tokens); // 释放字符串数组的内存

    return 0;
}

```

3.4.2 代码解释

1. **split 函数**：这个函数接受三个参数：需要分割的字符串 `str`，分隔符 `delimiter`，以及一个指向整数的指针 `count` 用于返回分割后的子字符串数量。函数返回一个指向字符串数组的指针，每个元素都是分割后的子字符串。
2. **内存管理**：函数内部使用 `strdup` 创建字符串的副本，因为 `strtok` 会修改原始字符串。每次分割出一个子字符串时，也使用 `strdup` 复制该子字符串，以便可以独立地释放它们。最后，确保释放所有分配的内存。
3. **动态数组**：使用动态数组 `tokens` 来存储分割后的子字符串指针。如果分割出的子字符串数量超过了当前数组的大小，会重新分配更大的内存。
4. **main 函数**：在 `main` 函数中调用 `split` 函数，并打印分割后的结果。最后，释放所有分配的内存。

3.5 链表逆序

3.5.1 链表逆序逻辑

(理解它的时候，需要自行画图，在面试笔试节点，直接死记硬背)

链表逆序的目的是将一个链表的顺序翻转，使得原本位于头部的节点移动到尾部，尾部的节点移动到头部。

这个过程可以通过迭代或递归实现，这里我们讲解迭代的方法。

迭代逆序链表的基本逻辑如下：

1. **初始化三个指针**：`prev`（前驱指针，初始为NULL），`curr`（当前指针，初始指向链表头节点），`next`（下一个指针，用于临时存储下一个节点）。
2. **遍历链表**：当 `curr` 不为空时，执行以下操作：
 - 将 `next` 指针指向 `curr->next`，即保存当前节点的下一个节点。
 - 将 `curr->next` 指向 `prev`，即翻转当前节点的指向，使其指向前一个节点。
 - 移动 `prev` 和 `curr` 指针：`prev` 向前移动到 `curr` 的位置，`curr` 向前移动到 `next` 的位置。
3. **更新头节点**：遍历结束后，`prev` 指针将指向新的头节点，因此将 `head` 更新为 `prev`。

3.5.2 逐行代码及注释

下面是链表逆序的函数实现，包含逐行代码和注释：

```
#include <stdio.h>
#include <stdlib.h>

typedef struct ListNode {
    int val;
    struct ListNode *next;
} ListNode;

/**
 * 逆序链表的函数
 * @param head 链表的头节点
 * @return 新链表的头节点
 */
ListNode* reverseList(ListNode* head) {
    // 初始化前驱指针为NULL
    ListNode *prev = NULL;
    // 初始化当前指针为头节点
    ListNode *curr = head;
    // 初始化下一个指针为NULL
    ListNode *next = NULL;

    // 遍历链表
    while (curr != NULL) {
        // 保存当前节点的下一个节点
        next = curr->next;
        // 翻转当前节点的指向，使其指向前一个节点
        curr->next = prev;
        // 前驱指针向前移动到当前节点
        prev = curr;
        // 当前指针向前移动到下一个节点
    }
}
```

```
    curr = next;
}
// 更新头节点为新的头节点
head = prev;
return head;
}
```

3.7 把数字转成字符串

在C语言中，将一个整数转换为字符串可以通过多种方法实现，这里提供一个简单的示例，使用 `sprintf` 函数将整数转换为字符串：

```
#include <stdio.h>

int main() {
    int num = 123; // 这是我们要转换的整数
    char str[50]; // 用于存储转换后的字符串，确保空间足够大

    // 使用sprintf函数将整数格式化为字符串
    sprintf(str, "%d", num);

    // 输出转换后的字符串
    printf("The integer as a string is: %s\n", str);

    return 0;
}
```

在上面的代码中，`sprintf` 函数将整数 `num` 格式化为字符串，并存储在 `str` 数组中。`%d` 是格式说明符，用于指定 `num` 是一个整数。

请注意，使用 `sprintf` 时需要确保目标字符串数组足够大，以避免溢出。在实际应用中，如果不确定需要多大的空间，可以使用 `snprintf` 函数，它允许你指定最大字符数，从而避免溢出。

这是一个使用 `snprintf` 的例子：

```
#include <stdio.h>

int main() {
    int num = 123;
    char str[50];

    // 使用snprintf，指定最大长度为sizeof(str) - 1，以确保不会溢出
    snprintf(str, sizeof(str), "%d", num);

    printf("The integer as a string is: %s\n", str);

    return 0;
}
```

在实际编程中，推荐使用 `snprintf` 来提高程序的安全性。

3.8 编写一个函数实现字符串的翻转

下面是一个简单的C语言函数，用于反转一个字符串：

```
#include <stdio.h>
#include <string.h>

void reverseString(char* str) {
    int len = strlen(str);
    for (int i = 0; i < len / 2; i++) {
        // 交换前后对应的字符
        char temp = str[i];
        str[i] = str[len - i - 1];
        str[len - i - 1] = temp;
    }
}

int main() {
    char myString[] = "Hello, world!";
    reverseString(myString);
    printf("Reversed String: %s\n", myString);
    return 0;
}
```

这个 `reverseString` 函数接受一个 `char` 类型的指针 `str` 作为参数，这个指针指向需要被反转的字符串。函数首先计算字符串的长度，然后使用一个循环交换字符串首尾对应的字符，直到到达字符串的中间位置。这样，字符串就被反转了。

在 `main` 函数中，我们创建了一个字符串 `myString`，调用 `reverseString` 函数来反转它，然后打印出反转后的结果。

3.9 指针定义汇总-笔试会出现

```
// 1. 一个整型数
int a;

// 2. 一个指向整型数的指针
int *a;

// 3. 一个指向指针的指针，它指向的指针是指向一个整型数
int **a;

// 4. 一个有10个整型数的数组
int a[10];

// 5. 一个有10个指针的数组，该指针是指向一个整型数
int *a[10];

// 6. 一个指向有10个整型数数组的指针
int (*a)[10];

// 7. 一个指向函数的指针，该函数有一个整型参数并返回一个整型数
int (*a)(int);

// 8. 一个有10个指针的数组，该指针指向一个函数，该函数有一个整型参数并返回一个整型数
```

```
int ((*a)[10])(int);
```

4. 其他

4.1 补码

补码 (Two's Complement) 是一种在计算机科学中常用的表示整数的方法，特别是在二进制计算机系统中。它主要用于简化二进制数的加减运算，使得加法可以用来执行减法。补码表示法在计算机中被广泛使用，因为它可以轻松地将加法和减法统一处理，并且可以方便地处理负数。

4.1.1 补码的定义

对于一个正整数，其补码就是其二进制表示。而对于一个负整数，其补码是该数的绝对值的二进制表示取反（即每个位上的1变成0，0变成1）后加1。

4.1.2 补码的优点

1. **简化运算**：使用补码，计算机可以只用加法器来执行加法和减法，简化了硬件设计。
2. **避免零的两种表示**：在没有补码的情况下，正零和负零可能会有不同的表示，使用补码后，零只有一种表示。
3. **溢出检测**：在补码表示法中，溢出可以通过简单的范围检查来确定。

4.1.3 补码的计算

假设我们有一个4位的二进制数，以下是如何计算正数和负数的补码的例子：

- **正数**：+5的二进制表示是0101。
- **负数**：-5的补码计算如下：
 - 首先找到5的二进制表示：0101
 - 然后取反得到：1010
 - 最后加1得到：1011

所以，-5在4位二进制补码表示中是1011。

4.1.4 溢出和范围

使用补码表示时，整数的范围取决于位数。例如，一个4位的补码系统可以表示的整数范围是-8到+7。如果进行加法或减法运算时结果超出了这个范围，就会发生溢出。

4.1.5 示例

让我们来看一个简单的补码加法的例子：

```
  0001  (+1)
+ 0011  (+3)
-----
  0110  (+4)
```

以上是加法的例子，很好理解，我们看个减法的案例，比如5-3，它被转换为加法：

步骤1：表示被减数和减数

- 被减数 (+5) 的4位二进制补码是 (0101)。

- 减数 (-3) 的4位二进制补码是 (1101)
 - 是这么得来的, -3是3的取反加一, 3是0011, 取反是1100, 再加1就是1101

步骤2: 将减法转换为加法

我们将 (5 - 3) 转换为 (5 + (-3)), 即:
[0101 + 1101]

步骤3: 执行加法

接下来, 我们执行这两个补码的加法运算:

```
  0101
+ 1101
-----
 10010
```

这里, 我们得到了一个5位的结果, 最左边的位是溢出位。

步骤4: 处理溢出

在4位二进制补码系统中, 我们只能保留4位的结果。因此, 我们丢弃最左边的溢出位 (1), 保留剩下的4位:

```
0010
```

步骤5: 解释结果

最后, 我们得到的结果是 (0010)。在4位补码系统中, (0010) 表示的是 (2), 这是 (5 - 3) 的正确结果。

总结

在4位二进制补码系统中, 当我们执行 (5 - 3) 的操作时, 我们实际上是在计算 (5 + (-3))。我们首先找到 (-3) 的补码, 然后将其加到 (5) 的补码上。得到的5位结果中, 我们丢弃最左边的溢出位, 保留剩下的4位, 这给出了正确的结果 (2)。

希望这次的解释清楚地说明了如何在4位补码系统中正确处理减法运算和溢出。

4.2 反码-了解一下

在计算机科学中, 反码是数值存储和运算的一种重要方式, 它主要用于简化某些特定的运算过程。以下是关于反码的定义、计算方法及其在加减运算中的应用的详细解释。

4.2.1 反码的定义

反码表示法规定:

- 正数的反码与其原码相同。
- 负数的反码是在其原码的基础上, 符号位保持不变 (仍为1), 其余各位按位取反 (即0变为1, 1变为0)。

4.2.2 反码的计算方法

1. **正数的反码**: 直接取其原码即可。
2. **负数的反码**: 在其原码的基础上, 除符号位外, 其余各位按位取反。

4.2.3 补码反码的关系

反码和补码都是计算机中用于表示整数的二进制编码方式，尤其是在表示负数时。它们之间的关系和区别如下：

反码 (One's Complement)

- **正数**：正数的反码与其原码相同，即直接将十进制数转换为二进制。
- **负数**：负数的反码是其正数对应值的二进制表示取反（按位取反）。

补码 (Two's Complement)

- **正数**：正数的补码与其原码相同，即直接将十进制数转换为二进制。
- **负数**：负数的补码是其正数对应值的二进制表示取反后加1。

反码和补码的关系

1. **转换关系**：一个数的补码可以通过对其反码加1得到，即：

$$\{\text{补码}\} = \{\text{反码}\} + 1$$

反之，一个数的反码可以通过对其补码减1得到，即：

$$\{\text{反码}\} = \{\text{补码}\} - 1$$

2. **零的表示**：

- 在反码表示中，零有两种表示：+0和-0。+0的反码是000...0，而-0的反码是111...1（所有位取反）。
- 在补码表示中，零只有一种表示：000...0。这是因为+0的补码是000...0，而-0的补码（即+0取反后加1）也是000...0。

3. **溢出处理**：

- 在反码表示中，最高位（符号位）的进位会被丢弃，这可能导致溢出。
- 在补码表示中，最高位的进位同样会被丢弃，但由于补码中零只有一种表示，因此溢出可以被检测。

4.3 结构体大小计算

计算结构体的大小涉及到了解编译器如何对结构体中的成员进行内存对齐。

4.3.1 结构体大小的影响因素

1. **成员变量的大小**：结构体中每个成员变量的大小直接影响结构体的总大小。
2. **内存对齐**：编译器为了提高访问效率，会按照特定的规则对结构体成员进行内存对齐。这意味着成员变量可能会有填充字节（padding）。
3. **填充 (Padding)**：填充字节是编译器在结构体成员之间或结构体尾部插入的未使用的字节，以满足对齐要求。
4. **编译器实现**：不同的编译器可能对内存对齐的处理方式不同，因此相同的结构体在不同的编译器或编译器设置下可能会有不同的大小。

4.3.2 计算结构体大小

在C语言中，可以使用 `sizeof` 运算符来获取结构体的大小。下面是一个例子：

```
#include <stdio.h>

typedef struct {
    char a; // 占用1字节
    int b;  // 占用4字节，但可能会有3字节的填充，以确保int是4字节对齐的
    char c; // 占用1字节
} ExampleStruct;

int main() {
    printf("Size of ExampleStruct: %lu bytes\n", sizeof(ExampleStruct));
    return 0;
}
```

在这个例子中，`ExampleStruct` 的实际大小取决于 `int` 的对齐要求。如果 `int` 需要4字节对齐，那么结构体的大小将是：

- `char a`：1字节
- 填充：3字节（为了使 `int b` 4字节对齐）
- `int b`：4字节
- `char c`：1字节
- 总大小：1 + 3 + 4 + 1 = 9字节，面试回答的时候12个字节，并说明原因

由于 `char c` 后面可能没有额外的空间，编译器可能会在 `char c` 后面再添加填充，以确保整个结构体的大小是4的倍数（如果编译器按照4字节对齐整个结构体）。因此，最终的大小可能是12字节。

4.3.3 强制结构体对齐

1. 使用 `#pragma pack`

`#pragma pack` 是一种跨平台的方法，可以在编译时指定结构体的对齐字节。

```
#include <stdio.h>

#pragma pack(push, 1) // 保存当前对齐设置，并设置对齐为1字节
typedef struct {
    char a; // 占用1字节
    int b;  // 占用4字节，没有额外的填充
    char c; // 占用1字节
} ExampleStruct;

#pragma pack(pop) // 恢复之前的对齐设置

int main() {
    printf("Size of ExampleStruct: %lu bytes\n", sizeof(ExampleStruct));
    return 0;
}
```

在这个例子中，`ExampleStruct` 的大小将是 `1 + 4 + 1 = 6` 字节，因为没有添加额外的填充。

2. 使用 `__attribute__((packed))`

在GCC编译器中，可以使用 `__attribute__((packed))` 来强制结构体不进行额外的对齐。

```
#include <stdio.h>

typedef struct __attribute__((packed)) {
    char a;    // 占用1字节
    int b;     // 占用4字节，没有额外的填充
    char c;    // 占用1字节
} ExampleStruct;

int main() {
    printf("Size of ExampleStruct: %lu bytes\n", sizeof(ExampleStruct));
    return 0;
}
```

这将导致 `ExampleStruct` 的大小为 6 字节，因为结构体成员将紧密排列，没有填充。

3. 使用 `#pragma pack` 指定对齐字节

你也可以指定特定的对齐字节数。

```
#include <stdio.h>

#pragma pack(2) // 设置结构体对齐为2字节
typedef struct {
    char a;    // 占用1字节，后面跟1字节填充
    int b;     // 占用4字节，后面跟2字节填充以满足2字节对齐
    char c;    // 占用1字节，后面跟1字节填充
} ExampleStruct;

#pragma pack() // 恢复默认对齐设置

int main() {
    printf("Size of ExampleStruct: %lu bytes\n", sizeof(ExampleStruct));
    return 0;
}
```

在这个例子中，`ExampleStruct` 的大小将是 8 字节，因为每个成员后面都有填充以满足2字节对齐。

注意事项

- 强制结构体对齐可能会影响程序的性能，因为对齐可以提高内存访问的速度。
- 强制对齐可能会增加结构体的大小，从而增加内存的使用。
- 在网络通信或文件I/O中，强制对齐有时是必要的，以确保数据的兼容性。

使用这些方法时，需要根据具体的编译器和平台选择合适的指令或关键字。

4.4 结构体位域

在C语言中，结构体位域（也称为位字段）是一种数据结构，它允许在结构体中为单个成员分配特定的位数。位域通常用于打包数据以节省空间，或者用于访问和控制硬件设备的寄存器。

4.4.1 位域的定义和使用

位域通过在结构体定义中指定成员的大小来创建。这可以通过在成员类型后添加冒号和位数来实现。

```
#include <stdio.h>

// 定义一个带有位域的结构体
typedef struct {
    unsigned int is_enabled : 1; // 1位
    unsigned int has_error : 1; // 1位
    unsigned int data_ready : 1; // 1位
    unsigned int mode : 2;      // 2位
    unsigned int : 2;           // 未使用的2位，用于填充
} DeviceStatus;

int main() {
    DeviceStatus dev;

    // 设置位域的值
    dev.is_enabled = 1;
    dev.has_error = 0;
    dev.data_ready = 1;
    dev.mode = 3; // 模式设置为二进制的11

    // 打印位域的值
    printf("is_enabled: %d\n", dev.is_enabled);
    printf("has_error: %d\n", dev.has_error);
    printf("data_ready: %d\n", dev.data_ready);
    printf("mode: %d\n", dev.mode);

    return 0;
}
```

4.4.2 位域的特性

1. **节省空间**：位域允许在结构体中紧凑地存储多个布尔值或小数值。
2. **硬件访问**：位域常用于嵌入式编程中，用于访问和控制硬件设备的寄存器。
3. **端口操作**：位域可以用于设置和清除特定的标志位，这在硬件编程中非常有用。

4.4.3 注意事项

- **可移植性**：位域的布局（即位域在内存中的排列）在不同的编译器和硬件架构中可能有所不同，这可能影响程序的可移植性。
- **性能**：虽然位域可以节省空间，但在某些架构上访问位域可能不如访问标准整数类型那样高效。
- **未使用的位**：可以在结构体中添加未使用的位来填充，以确保特定的对齐或布局。

4.4.4 位域的大小和对齐

位域的大小是由其声明的位数决定的，但是整个结构体的大小和对齐仍然受到编译器的控制。编译器可能会在结构体的末尾添加填充，以确保整个结构体的大小是某个特定值（如4字节）的倍数。

位域是C语言中一个强大的特性，特别是在需要节省空间或与硬件交互时。然而，使用位域时需要考虑到可移植性和性能问题。

4.5 联合体大小计算

在C语言中，联合体（Union）是一种特殊的数据类型，允许在相同的内存位置存储不同的数据类型。联合体的大小计算取决于其成员中最大的成员大小，因为联合体足够大以容纳其任何成员。

4.5.1 联合体的定义

联合体可以包含不同类型的成员，但任何时候只能使用其中一个成员。

```
#include <stdio.h>

typedef union {
    char a;        // 占用1字节
    int b;         // 占用4字节
    float c;       // 占用4字节，可能与int相同
    double d;      // 占用8字节
} ExampleUnion;
```

4.5.2 联合体大小的计算

联合体的大小是其所有成员中最大的成员的大小。在上述例子中，`double d` 是最大的成员，占用8字节，因此整个联合体的大小也将是8字节。

```
int main() {
    printf("Size of ExampleUnion: %lu bytes\n", sizeof(ExampleUnion));
    return 0;
}
```

4.5.3 联合体的特性

1. **内存重叠**：联合体的所有成员共享同一块内存空间，因此在同一时间只能使用一个成员。
2. **访问效率**：访问联合体成员的速度通常很快，因为所有成员都位于相同的内存地址。
3. **空间节省**：如果不同的数据类型只在不同的时间使用，联合体可以节省空间。

4.5.4 注意事项

- **内存对齐**：尽管联合体的大小由最大成员决定，但联合体的实际内存对齐可能受到编译器的影响。有些编译器可能会使联合体按照最大成员的对齐要求进行对齐。
- **未使用的位**：在某些情况下，编译器可能会在联合体的末尾添加填充，以确保整个联合体的大小符合特定的对齐要求。
- **可移植性**：不同编译器和平台上的联合体大小可能会有所不同，因此依赖于联合体大小的程序可能需要特别注意可移植性问题。

联合体是C语言中一个有用的特性，特别是在需要节省空间或需要在同一内存位置存储不同类型的数据时。然而，使用联合体时需要小心，以确保程序的正确性和可移植性。

4.6 大小端字节序

大小端字节序 (Endianness) 是指多字节数据 (如整数、浮点数等) 在计算机内存中的存储顺序。主要有两种类型的大小端字节序：大端 (Big-endian) 和小端 (Little-endian)。

4.6.1 大端字节序 (Big-endian)

在大端字节序中，多字节数据的最高有效字节 (MSB) 存储在最低的内存地址处，而最低有效字节 (LSB) 存储在最高的内存地址处。这种字节序有时被称为“网络字节序”，因为它是网络协议中使用的标准字节序。

例如，假设有一个16位的整数 `0x1234`，在大端字节序中的存储方式如下：

内存地址	数据
0x00	0x12
0x01	0x34

4.6.2 小端字节序 (Little-endian)

在小端字节序中，多字节数据的最低有效字节 (LSB) 存储在最低的内存地址处，而最高有效字节 (MSB) 存储在最高的内存地址处。

使用同样的16位整数 `0x1234`，在小端字节序中的存储方式如下：

内存地址	数据
0x00	0x34
0x01	0x12

4.6.3 大小端字节序的影响

大小端字节序的不同会导致相同的数据在不同的计算机架构之间传输时需要进行字节序转换。例如，如果一个在大端字节序计算机上创建的文件被传输到小端字节序计算机上，那么文件中的多字节数据需要被正确解释。

4.6.4 如何检测大小端

在C语言中，可以通过一个简单的宏来检测系统的字节序：

```
#include <stdio.h>

// 检测大小端的宏，这里好像不好背，可以花时间！课程讲的使用结构体来测试大小端
#define IS_BIG_ENDIAN ((unsigned char)1 == ((unsigned char *)&1)[0])
#define IS_LITTLE_ENDIAN ((unsigned char)1 == ((unsigned char *)&1)[sizeof(int) - 1])

int main() {
    if (IS_BIG_ENDIAN) {
        printf("System is big-endian.\n");
    } else {
        printf("System is little-endian.\n");
    }
    return 0;
}
```

这个程序利用了C语言的地址运算和强制类型转换来确定当前系统的字节序。如果 1 的最低位字节出现在地址的开始处，则系统是小端字节序；如果出现在地址的末尾，则系统是大端字节序。

用结构体的方式来测试大小端

```
#include <stdio.h>

// 定义一个结构体，其中包含一个char成员和一个int成员
typedef struct {
    char c; // 1字节
    int i; // 4字节（大多数现代系统）
} CheckEndian;

int main() {
    CheckEndian check;
    check.c = 1; // 将char成员设置为1，即内存中的最低有效字节为1

    // 根据int成员的值来判断字节序
    if (*(unsigned char *)&check.i == 1) {
        printf("Little-endian system.\n");
    } else {
        printf("Big-endian system.\n");
    }

    return 0;
}
```

在这个程序中，我们定义了一个名为 CheckEndian 的结构体，它包含一个 char 成员 c 和一个 int 成员 i。我们通过将 char 成员 c 设置为 1 来初始化结构体，这将导致结构体在内存中的最低有效字节（即 c 所占的字节）为 1。

接下来，我们通过将 int 成员 i 的地址转换为 unsigned char 指针，并解引用它来获取 int 成员的最低有效字节。如果这个字节是 1（即 char 成员 c 的值），那么系统就是小端字节序；否则，系统就是大端字节序。

4.6.6 字节序转换函数

在C语言库中，提供了一些函数来处理字节序转换，例如 htons()、htonl()、ntohs() 和 ntohl()，分别用于16位和32位整数的网络字节序与主机字节序之间的转换。

- htons()：将主机字节序的16位整数转换为网络字节序。
- htonl()：将主机字节序的32位整数转换为网络字节序。
- ntohs()：将网络字节序的16位整数转换为主机字节序。
- ntohl()：将网络字节序的32位整数转换为主机字节序。

这些函数定义在 <arpa/inet.h> 或 <netinet/in.h> 头文件中，通常在处理网络通信时使用。

4.7 C语言数据存放位置-重要-面试常问

在C语言中，数据存储的位置主要分为两种：栈（Stack）和堆（Heap）。以下是一些基本的规则来区分哪些数据存储在栈上，哪些存储在堆上：

4.7.1 存储在栈上的数据

1. **局部变量**：在函数内部定义的变量，它们在函数调用时被创建，并在函数返回时被销毁。
2. **函数参数**：传递给函数的参数通常存储在栈上。
3. **返回地址**：当函数调用发生时，返回地址也被存储在栈上，以便函数执行完毕后能够返回到正确的位置。
4. **基指针（Base Pointer）**：在一些架构中，用于跟踪栈帧的基指针也存储在栈上。

4.7.2 存储在堆上的数据

1. **动态分配的内存**：使用 `malloc`、`calloc`、`realloc` 或 `new` (C++) 分配的内存存储在堆上。这些内存必须显式地被释放（使用 `free` 或 `delete`）。
2. **全局变量和静态变量**：定义在函数外部的全局变量和静态变量存储在数据段，这通常被视为堆的一部分，但它们在整個程序的生命周期内都存在。
3. **常量**：在某些情况下，大的常量数据（如字符串字面量）可能会存储在数据段，这也被视为堆的一部分。

4.7.3 区别

- **生命周期**：栈上的数据在函数调用结束后自动销毁，而堆上的数据需要程序员手动管理其生命周期。
- **大小限制**：栈通常有大小限制，而堆的大小通常由操作系统和可用内存决定。
- **访问速度**：栈上的数据通常比堆上的数据访问更快，因为栈是连续的内存区域，而堆内存分配可能涉及更复杂的管理机制。

在C语言中，程序员需要了解这些存储区域，以便有效地管理内存和避免内存泄漏等问题。

4.8 引用和指针的区别

引用是C++的概念，这里有学过C++的同学可以记忆下，10种能回答1,2,3,4,5,6就行

1. **定义和初始化**：
 - 引用：在声明时必须被初始化，并且一旦初始化后不能改变其指向的目标。
 - 指针：可以在声明时初始化，也可以稍后被赋予新的地址，可以指向不同的变量，或者被设置为 `nullptr`。
2. **内存占用**：
 - 引用：不占用额外内存，它们只是另一个变量的别名。
 - 指针：占用内存来存储一个地址值。
3. **类型固定性**：
 - 引用：一旦引用被初始化为某个类型的变量，它就不能引用其他类型的变量。
 - 指针：虽然有类型的指针，但可以通过类型转换指向不同类型的数据。
4. **操作语法**：
 - 引用：使用引用时，不需要解引用操作符（*），直接使用变量名即可。
 - 指针：需要使用解引用操作符（*）来访问指针指向的值。

5. 与 `nullptr` 的关系:

- 引用: 引用不能被设置为 `nullptr`。
- 指针: 可以被显式设置为 `nullptr`, 表示没有指向任何对象。

6. 重定向能力:

- 引用: 一旦引用绑定到一个对象, 就不能重新绑定到另一个对象。
- 指针: 可以随时改变指向的目标。

7. 数组和函数参数:

- 引用: 不能直接用于数组 (可以使用数组引用, C++11 及以后版本支持), 在函数参数中使用引用可以避免复制, 并且保证对实际参数的操作。
- 指针: 可以直接用于数组, 并且常用于函数参数, 以传递动态分配的内存或者实现某些特定的功能。

8. 间接访问:

- 引用: 没有间接访问的概念, 引用本身就是目标对象的别名。
- 指针: 通过指针访问值时需要间接访问, 使用 `*pointer_name`。

9. 安全性:

- 引用: 更安全, 因为它们不能被重新绑定, 也不能被设置为 `nullptr`, 减少了空指针解引用的风险。
- 指针: 由于可以被设置为 `nullptr` 或重新指向, 需要更多的注意来避免空指针解引用和野指针问题。

10. 用途:

- 引用: 常用于函数参数和返回值, 以提供对实际参数的直接访问, 或者返回函数内部的局部变量的引用。
- 指针: 用于动态内存分配, 数组, 复杂的数据结构 (如链表、树等), 以及需要指针算术和间接访问的场合。

这些对比点概括了引用和指针在C++中的主要区别和它们的使用场景。